AML Transaction Monitoring Analytics — Full Study Pack

Generated from repository: C:\Users\diego\Desktop\Proyectos\AML\AML

1. Project Overview

This repository is a simplified AML transaction monitoring analytics project for a junior Data/BI + AML analytics portfolio. It uses synthetic account and transaction CSV files, cleans and standardizes them with Python/Pandas, creates basic behavioural features, applies explainable rule-based alert detection, and produces an alerts CSV for analyst review or dashboarding practice.

The main pipeline is:

1. src/ingest_data.py

2. src/clean_data.py

3. src/generate_features.py

4. src/detect_alerts.py

The main output is outputs/alerts_sample.csv. The project is educational, uses synthetic data, and does not make real compliance decisions.

Core project = simplified junior analytics project. experimental/ = optional, archived or future-work material.

2. Repository Map

The following tree includes files selected for this study pack:

```
.gitignore
README.md
data/
  processed/
    account_features.csv
    accounts_clean.csv
    accounts_ingested.csv
    transaction_features.csv
    transactions_clean.csv
    transactions_ingested.csv
  raw/
    accounts.csv
    transactions.csv
docs/
  case_studies.md
  export/
    README.md
    build_project_pdf.py
  methodology.md
experimental/
  README.md
  graph_analytics/
    README.md
    load_graph.py
  ml_scoring/
    README.md
    train_model.py
  orchestration/
    .env.example
    README.md
    dags/
      amlguardian_pipeline.py
    dbt/
      dbt_project.yml
      models/
        marts/
          alerts/
            alert_circular.sql
            alert_fan_in.sql
            alert_fan_out.sql
            alert_high_risk_geography.sql
            alert_layering.sql
            alert_smurfing.sql
            fct_alerts.sql
            fct_sar_candidates.sql
            schema.yml
          features/
            fct_account_features.sql
            schema.yml
        staging/
          schema.yml
          src_raw.yml
```

```
            stg_accounts.sql
            stg_transactions.sql
        profiles.yml
        tests/
          alerts_positive_amounts.sql
          features_counterparts_not_exceed_transactions.sql
          features_ratio_night_between_zero_and_one.sql
          sar_candidates_meet_threshold.sql
          transactions_amounts_non_negative.sql
          transactions_ingestion_not_before_event.sql
      docker/
        postgres/
          init/
            01_init.sql
            02_post_staging_indexes.sql
      docker-compose.yml
      legacy_postgres_ingestion/
        common.py
        ingest.py
    sql_investigations/
      investigations/
        circular_flows.sql
        fan_out_investigation.sql
        geographic_analysis.sql
        high_risk_accounts.sql
        sar_candidates.sql
        smurfing_deep_dive.sql
  outputs/
    alerts_sample.csv
  requirements.txt
  sql/
    01_create_tables.sql
    02_account_features.sql
    03_detection_rules.sql
  src/
    clean_data.py
    detect_alerts.py
    generate_features.py
    ingest_data.py
```

3. Execution Flow

Step 1: `src/ingest_data.py`

- Input files: data/raw/accounts.csv, data/raw/transactions.csv.

- Output files: data/processed/accounts_ingested.csv, data/processed/transactions_ingested.csv.

- Main transformations: standardizes column names and adds an ingestion timestamp.

- Why it matters: creates a reproducible first landing layer while keeping the project CSV/Pandas based.

Step 2: `src/clean_data.py`

- Input files: ingested account and transaction CSVs from data/processed/.

- Output files: data/processed/accounts_clean.csv, data/processed/transactions_clean.csv.

- Main transformations: renames key fields, parses timestamps, normalizes currencies, converts amounts to EUR, handles missing values, filters invalid rows and removes duplicates.

- Why it matters: creates consistent data for reliable features and rules.

Step 3: `src/generate_features.py`

- Input files: clean account and transaction CSVs.

- Output files: data/processed/account_features.csv, data/processed/transaction_features.csv.

- Main transformations: joins country context, calculates inbound/outbound totals, transaction counts, unique counterparties, country counts, average amount and a high-risk jurisdiction flag.

- Why it matters: turns raw transactions into analyst-friendly behavioural indicators.

Step 4: `src/detect_alerts.py`

- Input file: data/processed/transaction_features.csv.

- Output file: outputs/alerts_sample.csv.

- Main transformations: applies smurfing, fan-in, fan-out and high-risk geography rules.

- Why it matters: produces a final table that can be reviewed by an analyst or used in a dashboard.

4. Data Dictionary

The definitions below are generated from CSV headers and code context. When the repository does not define a business meaning explicitly, the description says it is inferred.

data/raw/accounts.csv

- Row count: 25

- Columns:

- account_id: Unique account identifier; inferred from file/code context. - name: Raw account name before cleaning. - country: Account country used for geography features. - account_type: Synthetic type/category of the account.

data/raw/transactions.csv

- Row count: 21

- Columns:

- transaction_id: Unique transaction identifier. - timestamp: Raw transaction timestamp. - sender_account_id: Account sending funds. - receiver_account_id: Account receiving funds. - amount: Raw transaction amount. - currency: Transaction currency. - transaction_type: Synthetic transaction category/type. - is_laundering: Synthetic label included in data; not used as real compliance evidence.

data/processed/account_features.csv

- Row count: 25

- Columns:

- account_id: Unique account identifier; inferred from file/code context. - account_name: Readable account/customer name after cleaning; inferred from file/code context. - country: Account country used for geography features. - account_type: Synthetic type/category of the account. - transaction_count_out: Count of outgoing transactions for the account. - total_outbound_amount: Total EUR sent by the account. - avg_outbound_amount: Average EUR amount for outgoing transactions. - unique_outbound_counterparties: Number of distinct receivers for outgoing transactions. - transaction_count_in: Count of incoming transactions for the account. - total_inbound_amount: Total EUR received by the account. - avg_inbound_amount: Average EUR amount for incoming transactions. - unique_inbound_counterparties: Number of distinct senders for incoming transactions. - unique_counterparties: Number of distinct counterparties connected to the account. - number_of_countries: Number of distinct counterparty countries. - high_risk_jurisdiction_flag: 1 when a transaction/account touches the training watchlist. - transaction_count: Total count of inbound and outbound transactions. - average_transaction_amount: Average transaction amount; inferred from generated features.

data/processed/accounts_clean.csv

- Row count: 25

- Columns:

- account_id: Unique account identifier; inferred from file/code context. - account_name: Readable account/customer name after cleaning; inferred from file/code context. - country: Account country used for geography features. - account_type: Synthetic type/category of the account.

data/processed/accounts_ingested.csv

- Row count: 25

- Columns:

- account_id: Unique account identifier; inferred from file/code context. - name: Raw account name before cleaning. - country: Account country used for geography features. - account_type: Synthetic type/category of the account. - ingested_at: Meaning inferred from file/code context; no richer definition is explicit in the repository.

data/processed/transaction_features.csv

- Row count: 21

- Columns:

- transaction_id: Unique transaction identifier. - transaction_timestamp: Cleaned timestamp parsed as UTC datetime. - transaction_date: Date derived from the transaction timestamp. - sender_account_id: Account sending funds. - receiver_account_id: Account receiving funds. - amount_original: Original transaction amount before conversion. - amount_eur: Transaction amount normalized to EUR in the simplified pipeline. - currency: Transaction currency. - transaction_type: Synthetic transaction category/type. - is_laundering: Synthetic label included in data; not used as real compliance evidence. - sender_country: Country joined from the sender account. - receiver_country: Country joined from the receiver account. - high_risk_jurisdiction_flag: 1 when a transaction/account touches the training watchlist.

data/processed/transactions_clean.csv

- Row count: 21

- Columns:

- transaction_id: Unique transaction identifier. - transaction_timestamp: Cleaned timestamp parsed as UTC datetime. - transaction_date: Date derived from the transaction timestamp. - sender_account_id: Account sending funds. - receiver_account_id: Account receiving funds. - amount_original: Original transaction amount before conversion. - amount_eur: Transaction amount normalized to EUR in the simplified pipeline. - currency: Transaction currency. - transaction_type: Synthetic transaction category/type. - is_laundering: Synthetic label included in data; not used as real compliance evidence.

data/processed/transactions_ingested.csv

- Row count: 21

- Columns:

- transaction_id: Unique transaction identifier. - timestamp: Raw transaction timestamp. - sender_account_id: Account sending funds. - receiver_account_id: Account receiving funds. - amount: Raw transaction amount. - currency: Transaction currency. - transaction_type: Synthetic transaction category/type. - is_laundering: Synthetic label included in data; not used as real compliance evidence. - ingested_at: Meaning inferred from file/code context; no richer definition is explicit in the repository.

outputs/alerts_sample.csv

- Row count: 6

- Columns:

- alert_id: Generated identifier for an alert. - account_id: Unique account identifier; inferred from file/code context. - rule_name: Name of the detection rule that created the alert. - severity: Simple priority label derived from count-based thresholds. - reason: Human-readable explanation of why the alert was generated. - metric_1_name: Name of the first supporting metric. - metric_1_value: Value of the first supporting metric. - metric_2_name: Name of the second supporting metric. - metric_2_value: Value of the second supporting metric. - detection_window_start: Start of the rule detection window. - detection_window_end: End of the rule detection window. - detection_date: Date associated with the alert window end.

5. Detection Rules Explained

Smurfing / Structuring

- Business intuition: repeated smaller outgoing transfers may indicate an attempt to split value into multiple payments.

- Technical logic: detect_smurfing checks outgoing transactions below EUR 9,999 and looks for at least 5 within a 72-hour window.

- Input columns: sender_account_id, amount_eur, transaction_timestamp.

- Output columns: alert_id, account_id, rule_name, severity, reason, supporting metric fields and detection window fields.

- Limitations: fixed thresholds, no customer profile, no historical baseline and no real compliance decision.

- Example alert: cfcd2d041e2f for account ACC-SMURF, severity MEDIUM. Reason: Account sent 5 transfers below EUR 9999 within 72 hours; this may indicate structuring and requires review.

Fan-In

- Business intuition: one account receiving funds from many unique originators in a short window may indicate collection-account behaviour.

- Technical logic: detect_fan_pattern groups by receiver_account_id and counts unique sender_account_id values within 24 hours.

- Input columns: receiver_account_id, sender_account_id, amount_eur, transaction_timestamp.

- Output columns: same alert schema as other rules.

- Limitations: can be normal for some business accounts; requires profile and activity-context review.

- Example alert: 315b28414d1a for account ACC-FANIN, severity MEDIUM. Reason: Account received funds from 5 unique counterparties within 24 hours; this may indicate unusual movement for review.

Fan-Out

- Business intuition: one account sending funds to many unique beneficiaries in a short window may indicate dispersion behaviour.

- Technical logic: detect_fan_pattern groups by sender_account_id and counts unique receiver_account_id values within 24 hours.

- Input columns: sender_account_id, receiver_account_id, amount_eur, transaction_timestamp.

- Output columns: same alert schema as other rules.

- Limitations: can be normal for payroll, refunds or supplier payments; further analyst review is required.

- Example alert: a814bec7113b for account ACC-FANOUT, severity MEDIUM. Reason: Account sent funds to 5 unique counterparties within 24 hours; this may indicate unusual movement for review.

High-Risk Geography

- Business intuition: activity involving a watchlist country may require extra context.

- Technical logic: detect_high_risk_geography checks whether sender or receiver country appears in the training watchlist: IRAN, MYANMAR, DPRK, SYRIA, YEMEN.

- Input columns: sender_country, receiver_country, amount_eur, account identifiers and timestamp.

- Output columns: same alert schema as other rules.

- Limitations: the country list is hardcoded for training and is not a maintained regulatory source.

- Example alert: fe7744a7b196 for account ACC-GEO, severity LOW. Reason: Account transacted with a training watchlist country (IRAN, MYANMAR); this requires further context review.

Circular Flow

- Business intuition: A -> B -> C -> A loops can indicate circular movement of funds.

- Technical logic: present as optional SQL practice in sql/03_detection_rules.sql, not as part of the core Python pipeline.

- Input columns: sender/receiver account IDs, transaction timestamps and EUR amount.

- Output columns: query result fields such as account path, time window and total loop amount.

- Limitations: marked as advanced practice; not a core junior-project detection rule.

6. Main Documentation Files

`README.md`

Main project description, positioning, run instructions and scope.

```
# AML Transaction Monitoring Analytics

AML Transaction Monitoring Analytics is a simplified AML transaction monitoring analytics project. It uses
synthetic transaction and account data to practice SQL, Python, alert logic and dashboarding in a realistic
junior data analytics portfolio context.

This is an educational portfolio project using synthetic data. It is not a production AML system and does not
make real compliance decisions.

One-sentence explanation:

> A simplified AML analytics project that uses Python and SQL to generate explainable rule-based alerts from
synthetic transaction data.

## Business Problem

Financial institutions need to monitor transactions and identify patterns that may indicate suspicious activity.
 In a real team, alerts would be reviewed by analysts together with KYC, customer context and internal
```

```
procedures.

This project focuses on the analytics layer: cleaning transaction data, creating basic behavioural features and
generating explainable alerts that could be used for investigation or dashboarding practice.

The repository is aimed at junior roles such as:

- Junior Data Analyst
- BI Analyst
- Risk / AML Analytics Junior
- Financial Crime Analytics Junior
- Data Quality / Data Management Junior

## What the Project Does

- Loads synthetic account and transaction CSV files.
- Cleans and standardizes column names, dates, currencies and missing values.
- Generates account-level and transaction-level features.
- Applies a small set of explainable rule-based detection scenarios.
- Produces an alerts CSV ready for analyst review or a simple dashboard.

## Detection Rules

The main project keeps the detection logic intentionally simple:

- **Smurfing / structuring:** several outgoing transfers below a threshold within a short time window.
- **Fan-in:** one account receives funds from several unique originators within 24 hours.
- **Fan-out:** one account sends funds to several unique beneficiaries within 24 hours.
- **High-risk geography:** an account transacts with a country included in a small watchlist for training
purposes.
- **Optional advanced SQL:** a circular flow example is included in SQL as future practice, not as part of the
core Python pipeline.

Each alert includes an `alert_id`, `account_id`, `rule_name`, `severity`, `reason`, supporting metrics and a
detection date or window.

## Tech Stack

Main stack:

- Python
- Pandas
- SQL
- CSV files for input/output

Optional or archived work is kept under `experimental/` for future learning. It is not required to run the
simplified project.

## Repository Layout
```

AML/ ■■■ README.md ■■■ requirements.txt ■■■ data/ ■ ■■■ raw/ ■ ■■■ processed/ ■■■ src/ ■ ■■■ ingest_data.py ■ ■■■ clean_data.py ■ ■■■ generate_features.py ■ ■■■ detect_alerts.py ■■■ sql/ ■ ■■■ 01_create_tables.sql ■ ■■■ 02_account_features.sql ■ ■■■ 03_detection_rules.sql ■■■ outputs/ ■ ■■■ alerts_sample.csv ■■■ docs/ ■ ■■■ methodology.md ■ ■■■ case_studies.md ■■■ experimental/ ■■■ ml_scoring/ ■■■ graph_analytics/ ■■■ orchestration/ ■■■ archive_docs/

```
## How to Run

Install dependencies:
```

pip install -r requirements.txt

```
Run the simple pipeline:
```

python src/ingest_data.py python src/clean_data.py python src/generate_features.py python src/detect_alerts.py

```
Input files:

- `data/raw/accounts.csv`
- `data/raw/transactions.csv`

Generated files:

- `data/processed/accounts_ingested.csv`
- `data/processed/transactions_ingested.csv`
- `data/processed/accounts_clean.csv`
- `data/processed/transactions_clean.csv`
- `data/processed/account_features.csv`
- `data/processed/transaction_features.csv`
- `outputs/alerts_sample.csv`

## Outputs

The final alerts table contains:

- alert identifier
- account identifier
- rule name
- severity
- short reason for review
- supporting metrics
```

```
- detection window

The alerts are not final conclusions. They show patterns that may indicate unusual behaviour and would require
further review by an analyst.

## Case Studies

### Case 1: Repeated Smaller Outbound Transfers

An account receives a larger inbound transfer and then sends several smaller payments to different beneficiaries
 over the next two days. The smurfing rule may indicate possible structuring because the payments are split into
 multiple lower-value transfers.

This would be escalated to an analyst for review of customer profile, expected activity and relationship with
the beneficiaries.

### Case 2: Collection Account Behaviour

Another account receives payments from several unrelated originators within a short time window. The fan-in rule
 may indicate that the account is acting as a collection point.

The alert does not prove suspicious activity. It highlights a pattern that requires further review, especially
if the account profile does not explain that volume or number of counterparties.

More detailed examples are available in `docs/case_studies.md`.

## Future Improvements

The following ideas are intentionally outside the main junior-friendly path:

- Airflow orchestration for scheduled runs.
- Graph analytics for network exploration.
- ML risk scoring as an optional experiment.
- Model explanation techniques as future learning.
- Analyst feedback loop to mark reviewed alerts and improve rules.

Advanced or previous work has been moved to `experimental/` so the main project stays easy to understand.
```

`docs/case_studies.md`

Provides realistic junior analyst case narratives.

```
# Case Studies

Los siguientes casos estan escritos como ejemplos de analisis junior. No son conclusiones regulatorias.

## Caso 1: Posible Structuring

**Cuenta:** `ACC-SMURF`
**Regla:** `SMURFING`
**Ventana:** 72 horas

La cuenta recibe un ingreso de mayor importe y despues realiza varios pagos inferiores a EUR 9,999 hacia
diferentes beneficiarios. Este patron puede indicar fragmentacion de pagos, aunque tambien podria tener una
explicacion comercial o personal legitima.

**Metricas a revisar:**

- numero de pagos salientes en 72 horas
- importe total enviado
- numero de beneficiarios distintos
- relacion entre el ingreso inicial y las salidas posteriores

**Siguiente paso analitico:**

El caso se escalaria a un analista para revisar el perfil esperado de la cuenta, la relacion con los
beneficiarios y si el comportamiento es habitual para ese cliente.

## Caso 2: Cuenta de Coleccion

**Cuenta:** `ACC-FANIN`
**Regla:** `FAN_IN`
**Ventana:** 24 horas

La cuenta recibe fondos desde varias contrapartes distintas en un mismo dia. Este comportamiento puede ser
normal en una cuenta de negocio, pero tambien puede indicar que la cuenta esta actuando como punto de
concentracion.

**Metricas a revisar:**

- numero de originadores unicos
- importe total recibido
- frecuencia de eventos similares
- tipo de cuenta y actividad esperada

**Siguiente paso analitico:**

La alerta requiere revisar si la actividad encaja con el perfil del cliente. Si no encaja, podria priorizarse
para investigacion adicional.

## Caso 3: Exposicion Geografica

**Cuenta:** `ACC-GEO`
**Regla:** `HIGH_RISK_GEOGRAPHY`
```

```
La cuenta tiene transacciones con contrapartes de paises incluidos en una pequena lista de entrenamiento. La
alerta no significa que la actividad sea irregular por si sola; solo indica que el pais de la contraparte
aumenta la necesidad de contexto.

**Metricas a revisar:**

- paises involucrados
- importe total
- direccion del dinero
- historial previo con esas contrapartes

**Siguiente paso analitico:**

El caso se revisaria junto con informacion del cliente, proposito de la relacion y documentacion disponible.
```

`docs/export/README.md`

Relevant repository text file included for study context.

```
# Project Study Pack Export

Regenerate the NotebookLM study pack from the repository root:
```

python docs/export/build_project_pdf.py

```
Outputs:

- `docs/export/AML_project_full_study_pack.md`
- `docs/export/AML_project_full_study_pack.pdf`

The export is intentionally focused on the simplified AML analytics project. Advanced files in `experimental/`
are included as optional/non-core context, while heavy folders and binary artifacts are skipped.
```

`docs/methodology.md`

Explains the project methodology in study-friendly language.

```
# Metodologia

Este proyecto usa datos sinteticos para practicar un flujo sencillo de analitica AML. No es un sistema de
cumplimiento real y no toma decisiones regulatorias.

## 1. Ingestion

Los ficheros `data/raw/accounts.csv` y `data/raw/transactions.csv` se cargan con Pandas desde
`src/ingest_data.py`. El objetivo es mantener una entrada clara y facil de revisar:

- cuentas
- transacciones
- importes
- divisas
- paises
- tipo de cuenta

## 2. Limpieza

`src/clean_data.py` aplica transformaciones basicas:

- estandarizacion de nombres de columnas
- conversion de fechas
- normalizacion de divisas
- conversion de importes a euros
- eliminacion de duplicados
- tratamiento de valores nulos sencillos

## 3. Generacion de Features

`src/generate_features.py` crea variables utiles para un analista junior:

- numero de transacciones
- total recibido
- total enviado
- numero de contrapartes unicas
- numero de paises relacionados
- importe medio por transaccion
- indicador de jurisdiccion de mayor riesgo segun una lista de entrenamiento

Estas features no son una evaluacion final del cliente. Sirven para explicar patrones y preparar una tabla para
analisis o dashboarding.

## 4. Reglas de Alertas

`src/detect_alerts.py` aplica reglas simples y explicables:

- smurfing / structuring
- fan-in
- fan-out
- geografia de mayor riesgo

Cada alerta incluye una razon y metricas de soporte. El lenguaje es intencionalmente prudente: una alerta puede
indicar un patron inusual, pero requiere revision humana y mas contexto.
## 5. Salida Analitica
```

```
La salida principal es `outputs/alerts_sample.csv`. Puede usarse para:

- practicar SQL
- construir un dashboard sencillo
- preparar historias de investigacion para entrevista
- explicar como se priorizan patrones para revision

## Limitaciones

- Los datos son sinteticos y pequenos.
- Las reglas tienen umbrales fijos.
- No hay KYC real, investigacion documental ni feedback de analistas.
- La lista de paises es solo una lista de entrenamiento.
- El proyecto no genera decisiones de cumplimiento.
```

7. Source Code

`requirements.txt`

Minimal Python dependencies for the simplified pipeline.

```
pandas==2.1.4
numpy==1.26.4
```

`src/clean_data.py`

Cleans accounts and transactions, parses timestamps and normalizes amounts.

```
from __future__ import annotations

from pathlib import Path

import pandas as pd


PROJECT_ROOT = Path(__file__).resolve().parents[1]
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"

FX_RATES_TO_EUR = {
    "EUR": 1.0,
    "USD": 0.92,
    "GBP": 1.17,
}


def clean_accounts(accounts: pd.DataFrame) -> pd.DataFrame:
    accounts = accounts.copy()
    accounts = accounts.rename(columns={"name": "account_name"})

    required = ["account_id", "account_name", "country", "account_type"]
    missing = [column for column in required if column not in accounts.columns]
    if missing:
        raise ValueError(f"Missing account columns: {missing}")

    accounts["account_id"] = accounts["account_id"].astype(str).str.strip()
    accounts["account_name"] = accounts["account_name"].fillna("UNKNOWN").astype(str).str.strip()
    accounts["country"] = accounts["country"].fillna("UNKNOWN").astype(str).str.upper().str.strip()
    accounts["account_type"] = accounts["account_type"].fillna("UNKNOWN").astype(str).str.upper().str.strip()

    accounts = accounts[accounts["account_id"] != ""]
    accounts = accounts.drop_duplicates(subset=["account_id"], keep="first")
    return accounts[required]


def clean_transactions(transactions: pd.DataFrame) -> pd.DataFrame:
    transactions = transactions.copy()
    transactions = transactions.rename(
        columns={
            "timestamp": "transaction_timestamp",
            "amount": "amount_original",
        }
    )

    required = [
        "transaction_id",
        "transaction_timestamp",
        "sender_account_id",
        "receiver_account_id",
        "amount_original",
        "currency",
        "transaction_type",
    ]
    missing = [column for column in required if column not in transactions.columns]
    if missing:
        raise ValueError(f"Missing transaction columns: {missing}")

    text_columns = ["transaction_id", "sender_account_id", "receiver_account_id"]
    for column in text_columns:
        transactions[column] = transactions[column].astype(str).str.strip()

    transactions["currency"] = transactions["currency"].fillna("EUR").astype(str).str.upper().str.strip()
```

```
    transactions["transaction_type"] = (
        transactions["transaction_type"].fillna("UNKNOWN").astype(str).str.upper().str.strip()
    )
    transactions["amount_original"] = pd.to_numeric(transactions["amount_original"], errors="coerce")
    transactions["amount_eur"] = (
        transactions["amount_original"] * transactions["currency"].map(FX_RATES_TO_EUR).fillna(1.0)
    ).round(2)
    transactions["transaction_timestamp"] = pd.to_datetime(
        transactions["transaction_timestamp"], errors="coerce", utc=True
    )
    transactions["transaction_date"] = transactions["transaction_timestamp"].dt.date.astype(str)

    if "is_laundering" in transactions.columns:
        transactions["is_laundering"] = (
            pd.to_numeric(transactions["is_laundering"], errors="coerce").fillna(0).astype(int)
        )
    else:
        transactions["is_laundering"] = 0

    before = len(transactions)
    transactions = transactions.dropna(subset=["transaction_timestamp", "amount_eur"])
    transactions = transactions[
        (transactions["transaction_id"] != "")
        & (transactions["sender_account_id"] != "")
        & (transactions["receiver_account_id"] != "")
        & (transactions["amount_eur"] > 0)
    ]
    transactions = transactions.drop_duplicates(subset=["transaction_id"], keep="first")
    dropped = before - len(transactions)
    if dropped:
        print(f"Dropped {dropped} invalid or duplicate transactions.")

    return transactions[
        [
            "transaction_id",
            "transaction_timestamp",
            "transaction_date",
            "sender_account_id",
            "receiver_account_id",
            "amount_original",
            "amount_eur",
            "currency",
            "transaction_type",
            "is_laundering",
        ]
    ]


def main() -> None:
    accounts = pd.read_csv(PROCESSED_DIR / "accounts_ingested.csv")
    transactions = pd.read_csv(PROCESSED_DIR / "transactions_ingested.csv")

    clean_accounts(accounts).to_csv(PROCESSED_DIR / "accounts_clean.csv", index=False)
    clean_transactions(transactions).to_csv(PROCESSED_DIR / "transactions_clean.csv", index=False)

    print("Clean data saved in data/processed/.")


if __name__ == "__main__":
    main()
```

`src/detect_alerts.py`

Applies simplified rule-based AML alert logic and writes the alerts output.

```
from __future__ import annotations

import hashlib
from pathlib import Path

import pandas as pd


PROJECT_ROOT = Path(__file__).resolve().parents[1]
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
OUTPUT_DIR = PROJECT_ROOT / "outputs"

SMURFING_THRESHOLD = 9_999
SMURFING_MIN_TXNS = 5
FAN_MIN_COUNTERPARTIES = 5

HIGH_RISK_COUNTRIES = {"IRAN", "MYANMAR", "DPRK", "SYRIA", "YEMEN"}


def make_alert_id(account_id: str, rule_name: str, window_end: pd.Timestamp) -> str:
    raw = f"{account_id}|{rule_name}|{window_end.isoformat()}"
    return hashlib.md5(raw.encode("utf-8")).hexdigest()[:12]


def severity_from_count(count: int) -> str:
    if count >= 10:
        return "HIGH"
    if count >= 5:
        return "MEDIUM"
```

```
    return "LOW"


def build_alert(
    account_id: str,
    rule_name: str,
    severity: str,
    reason: str,
    metric_1_name: str,
    metric_1_value: object,
    metric_2_name: str,
    metric_2_value: object,
    window_start: pd.Timestamp,
    window_end: pd.Timestamp,
) -> dict[str, object]:
    return {
        "alert_id": make_alert_id(account_id, rule_name, window_end),
        "account_id": account_id,
        "rule_name": rule_name,
        "severity": severity,
        "reason": reason,
        "metric_1_name": metric_1_name,
        "metric_1_value": metric_1_value,
        "metric_2_name": metric_2_name,
        "metric_2_value": metric_2_value,
        "detection_window_start": window_start.isoformat(),
        "detection_window_end": window_end.isoformat(),
        "detection_date": window_end.date().isoformat(),
    }


def detect_smurfing(transactions: pd.DataFrame) -> list[dict[str, object]]:
    alerts = []
    candidates = transactions[transactions["amount_eur"] < SMURFING_THRESHOLD].copy()
    for account_id, group in candidates.groupby("sender_account_id"):
        group = group.sort_values("transaction_timestamp")
        best_window = None
        for _, row in group.iterrows():
            window_start = row["transaction_timestamp"]
            window_end = window_start + pd.Timedelta(hours=72)
            window = group[
                (group["transaction_timestamp"] >= window_start)
                & (group["transaction_timestamp"] <= window_end)
            ]
            if len(window) >= SMURFING_MIN_TXNS:
                total_amount = round(window["amount_eur"].sum(), 2)
                candidate = (len(window), total_amount, window_start, window["transaction_timestamp"].max())
                if best_window is None or candidate[:2] > best_window[:2]:
                    best_window = candidate
        if best_window:
            count, total_amount, window_start, window_end = best_window
            alerts.append(
                build_alert(
                    account_id=account_id,
                    rule_name="SMURFING",
                    severity=severity_from_count(count),
                    reason=(
                        f"Account sent {count} transfers below EUR {SMURFING_THRESHOLD} "
                        "within 72 hours; this may indicate structuring and requires review."
                    ),
                    metric_1_name="transaction_count_72h",
                    metric_1_value=count,
                    metric_2_name="amount_72h_eur",
                    metric_2_value=total_amount,
                    window_start=window_start,
                    window_end=window_end,
                )
            )
    return alerts


def detect_fan_pattern(
    transactions: pd.DataFrame,
    account_column: str,
    counterparty_column: str,
    rule_name: str,
    direction_label: str,
) -> list[dict[str, object]]:
    alerts = []
    for account_id, group in transactions.groupby(account_column):
        group = group.sort_values("transaction_timestamp")
        best_window = None
        for _, row in group.iterrows():
            window_start = row["transaction_timestamp"]
            window_end = window_start + pd.Timedelta(hours=24)
            window = group[
                (group["transaction_timestamp"] >= window_start)
                & (group["transaction_timestamp"] <= window_end)
            ]
            unique_counterparties = window[counterparty_column].nunique()
            if unique_counterparties >= FAN_MIN_COUNTERPARTIES:
                total_amount = round(window["amount_eur"].sum(), 2)
                candidate = (
                    unique_counterparties,
                    total_amount,
```

```
                    window_start,
                    window["transaction_timestamp"].max(),
                )
                if best_window is None or candidate[:2] > best_window[:2]:
                    best_window = candidate
        if best_window:
            unique_counterparties, total_amount, window_start, window_end = best_window
            alerts.append(
                build_alert(
                    account_id=account_id,
                    rule_name=rule_name,
                    severity=severity_from_count(unique_counterparties),
                    reason=(
                        f"Account {direction_label} {unique_counterparties} unique counterparties "
                        "within 24 hours; this may indicate unusual movement for review."
                    ),
                    metric_1_name="unique_counterparties_24h",
                    metric_1_value=unique_counterparties,
                    metric_2_name="amount_24h_eur",
                    metric_2_value=total_amount,
                    window_start=window_start,
                    window_end=window_end,
                )
            )
    return alerts


def detect_high_risk_geography(transactions: pd.DataFrame) -> list[dict[str, object]]:
    alerts = []
    flagged = transactions[
        transactions["sender_country"].isin(HIGH_RISK_COUNTRIES)
        | transactions["receiver_country"].isin(HIGH_RISK_COUNTRIES)
    ].copy()
    if flagged.empty:
        return alerts

    account_rows = pd.concat(
        [
            flagged.rename(columns={"sender_account_id": "account_id"})[
                ["account_id", "transaction_timestamp", "amount_eur", "sender_country", "receiver_country"]
            ],
            flagged.rename(columns={"receiver_account_id": "account_id"})[
                ["account_id", "transaction_timestamp", "amount_eur", "sender_country", "receiver_country"]
            ],
        ],
        ignore_index=True,
    )

    for account_id, group in account_rows.groupby("account_id"):
        risky_countries = sorted(
            set(group["sender_country"]).union(set(group["receiver_country"])) & HIGH_RISK_COUNTRIES
        )
        window_start = group["transaction_timestamp"].min()
        window_end = group["transaction_timestamp"].max()
        exposure_count = len(group)
        alerts.append(
            build_alert(
                account_id=account_id,
                rule_name="HIGH_RISK_GEOGRAPHY",
                severity=severity_from_count(exposure_count),
                reason=(
                    "Account transacted with a training watchlist country "
                    f"({', '.join(risky_countries)}); this requires further context review."
                ),
                metric_1_name="high_risk_transfer_count",
                metric_1_value=exposure_count,
                metric_2_name="high_risk_amount_eur",
                metric_2_value=round(group["amount_eur"].sum(), 2),
                window_start=window_start,
                window_end=window_end,
            )
        )
    return alerts


def main() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    transactions = pd.read_csv(PROCESSED_DIR / "transaction_features.csv")
    transactions["transaction_timestamp"] = pd.to_datetime(
        transactions["transaction_timestamp"], utc=True
    )

    alerts = []
    alerts.extend(detect_smurfing(transactions))
    alerts.extend(
        detect_fan_pattern(
            transactions,
            account_column="receiver_account_id",
            counterparty_column="sender_account_id",
            rule_name="FAN_IN",
            direction_label="received funds from",
        )
    )
    alerts.extend(
        detect_fan_pattern(
```

```
            transactions,
            account_column="sender_account_id",
            counterparty_column="receiver_account_id",
            rule_name="FAN_OUT",
            direction_label="sent funds to",
        )
    )
    alerts.extend(detect_high_risk_geography(transactions))

    alerts_df = pd.DataFrame(alerts)
    alerts_df = alerts_df.sort_values(["severity", "rule_name", "account_id"])
    alerts_df.to_csv(OUTPUT_DIR / "alerts_sample.csv", index=False)

    print(f"Generated {len(alerts_df)} alerts in outputs/alerts_sample.csv.")


if __name__ == "__main__":
    main()
```

`src/generate_features.py`

Builds transaction-level and account-level analytical features.

```
from __future__ import annotations

from pathlib import Path

import pandas as pd


PROJECT_ROOT = Path(__file__).resolve().parents[1]
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"

HIGH_RISK_COUNTRIES = {"IRAN", "MYANMAR", "DPRK", "SYRIA", "YEMEN"}


def add_country_context(transactions: pd.DataFrame, accounts: pd.DataFrame) -> pd.DataFrame:
    account_countries = accounts[["account_id", "country"]]
    transactions = transactions.merge(
        account_countries.rename(columns={"account_id": "sender_account_id", "country": "sender_country"}),
        on="sender_account_id",
        how="left",
    )
    transactions = transactions.merge(
        account_countries.rename(columns={"account_id": "receiver_account_id", "country": "receiver_country"}),
        on="receiver_account_id",
        how="left",
    )
    transactions["sender_country"] = transactions["sender_country"].fillna("UNKNOWN")
    transactions["receiver_country"] = transactions["receiver_country"].fillna("UNKNOWN")
    transactions["high_risk_jurisdiction_flag"] = (
        transactions["sender_country"].isin(HIGH_RISK_COUNTRIES)
        | transactions["receiver_country"].isin(HIGH_RISK_COUNTRIES)
    ).astype(int)
    return transactions


def build_account_features(transactions: pd.DataFrame, accounts: pd.DataFrame) -> pd.DataFrame:
    outbound = transactions.groupby("sender_account_id").agg(
        transaction_count_out=("transaction_id", "count"),
        total_outbound_amount=("amount_eur", "sum"),
        avg_outbound_amount=("amount_eur", "mean"),
        unique_outbound_counterparties=("receiver_account_id", "nunique"),
    )
    inbound = transactions.groupby("receiver_account_id").agg(
        transaction_count_in=("transaction_id", "count"),
        total_inbound_amount=("amount_eur", "sum"),
        avg_inbound_amount=("amount_eur", "mean"),
        unique_inbound_counterparties=("sender_account_id", "nunique"),
    )

    sent_counterparties = transactions.rename(
        columns={
            "sender_account_id": "account_id",
            "receiver_account_id": "counterparty_account_id",
            "receiver_country": "counterparty_country",
        }
    )[["account_id", "counterparty_account_id", "counterparty_country", "high_risk_jurisdiction_flag"]]
    received_counterparties = transactions.rename(
        columns={
            "receiver_account_id": "account_id",
            "sender_account_id": "counterparty_account_id",
            "sender_country": "counterparty_country",
        }
    )[["account_id", "counterparty_account_id", "counterparty_country", "high_risk_jurisdiction_flag"]]
    counterparties = pd.concat([sent_counterparties, received_counterparties], ignore_index=True)
    counterpart_features = counterparties.groupby("account_id").agg(
        unique_counterparties=("counterparty_account_id", "nunique"),
        number_of_countries=("counterparty_country", "nunique"),
        high_risk_jurisdiction_flag=("high_risk_jurisdiction_flag", "max"),
    )

    features = accounts.set_index("account_id").join([outbound, inbound, counterpart_features])
    numeric_columns = [
```

```
        "transaction_count_out",
        "total_outbound_amount",
        "avg_outbound_amount",
        "unique_outbound_counterparties",
        "transaction_count_in",
        "total_inbound_amount",
        "avg_inbound_amount",
        "unique_inbound_counterparties",
        "unique_counterparties",
        "number_of_countries",
        "high_risk_jurisdiction_flag",
    ]
    features[numeric_columns] = features[numeric_columns].fillna(0)
    features["transaction_count"] = (
        features["transaction_count_out"] + features["transaction_count_in"]
    ).astype(int)
    features["average_transaction_amount"] = (
        (features["total_outbound_amount"] + features["total_inbound_amount"])
        / features["transaction_count"].replace(0, pd.NA)
    ).fillna(0)

    amount_columns = [
        "total_outbound_amount",
        "avg_outbound_amount",
        "total_inbound_amount",
        "avg_inbound_amount",
        "average_transaction_amount",
    ]
    features[amount_columns] = features[amount_columns].round(2)
    return features.reset_index()


def main() -> None:
    accounts = pd.read_csv(PROCESSED_DIR / "accounts_clean.csv")
    transactions = pd.read_csv(PROCESSED_DIR / "transactions_clean.csv")

    transaction_features = add_country_context(transactions, accounts)
    account_features = build_account_features(transaction_features, accounts)

    transaction_features.to_csv(PROCESSED_DIR / "transaction_features.csv", index=False)
    account_features.to_csv(PROCESSED_DIR / "account_features.csv", index=False)

    print(f"Generated features for {len(account_features)} accounts.")


if __name__ == "__main__":
    main()
```

`src/ingest_data.py`

Loads raw CSV files, standardizes headers and writes the ingested layer.

```
from __future__ import annotations

from datetime import datetime, timezone
from pathlib import Path

import pandas as pd


PROJECT_ROOT = Path(__file__).resolve().parents[1]
RAW_DIR = PROJECT_ROOT / "data" / "raw"
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"


def normalize_columns(dataframe: pd.DataFrame) -> pd.DataFrame:
    dataframe = dataframe.copy()
    dataframe.columns = (
        dataframe.columns.str.strip()
        .str.lower()
        .str.replace(r"[^a-z0-9]+", "_", regex=True)
        .str.strip("_")
    )
    return dataframe


def read_csv(name: str) -> pd.DataFrame:
    path = RAW_DIR / name
    if not path.exists():
        raise FileNotFoundError(f"Missing input file: {path}")
    return normalize_columns(pd.read_csv(path))


def main() -> None:
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
    ingested_at = datetime.now(timezone.utc).isoformat()

    accounts = read_csv("accounts.csv")
    transactions = read_csv("transactions.csv")

    accounts["ingested_at"] = ingested_at
    transactions["ingested_at"] = ingested_at

    accounts.to_csv(PROCESSED_DIR / "accounts_ingested.csv", index=False)
```

```
    transactions.to_csv(PROCESSED_DIR / "transactions_ingested.csv", index=False)

    print(f"Loaded {len(accounts)} accounts and {len(transactions)} transactions.")


if __name__ == "__main__":
    main()
```

`sql/01_create_tables.sql`

Creates simple SQL tables for accounts and transactions.

```
-- Simple SQLite-friendly schema for practicing the project with SQL.

DROP TABLE IF EXISTS accounts;
DROP TABLE IF EXISTS transactions;

CREATE TABLE accounts (
    account_id TEXT PRIMARY KEY,
    account_name TEXT,
    country TEXT,
    account_type TEXT
);

CREATE TABLE transactions (
    transaction_id TEXT PRIMARY KEY,
    transaction_timestamp TEXT,
    transaction_date TEXT,
    sender_account_id TEXT,
    receiver_account_id TEXT,
    amount_original REAL,
    amount_eur REAL,
    currency TEXT,
    transaction_type TEXT,
    is_laundering INTEGER
);
```

`sql/02_account_features.sql`

Shows SQL logic for account-level features.

```
-- Account-level features for analyst review and dashboarding.

WITH account_transactions AS (
    SELECT
        sender_account_id AS account_id,
        receiver_account_id AS counterparty_account_id,
        amount_eur,
        'OUTBOUND' AS direction
    FROM transactions

    UNION ALL

    SELECT
        receiver_account_id AS account_id,
        sender_account_id AS counterparty_account_id,
        amount_eur,
        'INBOUND' AS direction
    FROM transactions
)
SELECT
    a.account_id,
    a.country,
    COUNT(at.account_id) AS transaction_count,
    ROUND(SUM(CASE WHEN at.direction = 'INBOUND' THEN at.amount_eur ELSE 0 END), 2) AS total_inbound_amount,
    ROUND(SUM(CASE WHEN at.direction = 'OUTBOUND' THEN at.amount_eur ELSE 0 END), 2) AS total_outbound_amount,
    COUNT(DISTINCT at.counterparty_account_id) AS unique_counterparties,
    ROUND(AVG(at.amount_eur), 2) AS average_transaction_amount
FROM accounts a
LEFT JOIN account_transactions at
    ON a.account_id = at.account_id
GROUP BY a.account_id, a.country;
```

`sql/03_detection_rules.sql`

Shows SQL examples for simplified detection rules and optional circular flow.

```
-- Simplified detection rules for practice.
-- These queries are examples; the Python scripts generate the sample output CSV.

-- 1. Smurfing / structuring: several lower-value outgoing transfers in 72 hours.
SELECT
    t1.sender_account_id AS account_id,
    'SMURFING' AS rule_name,
    COUNT(*) AS transaction_count_72h,
    ROUND(SUM(t2.amount_eur), 2) AS amount_72h_eur,
    MIN(t2.transaction_timestamp) AS window_start,
    MAX(t2.transaction_timestamp) AS window_end
FROM transactions t1
JOIN transactions t2
    ON t1.sender_account_id = t2.sender_account_id
   AND t2.transaction_timestamp BETWEEN t1.transaction_timestamp
```

```
       AND datetime(t1.transaction_timestamp, '+72 hours')
WHERE t2.amount_eur < 9999
GROUP BY t1.sender_account_id, t1.transaction_timestamp
HAVING COUNT(*) >= 5;

-- 2. Fan-in: one account receives funds from several unique originators in 24 hours.
SELECT
    t1.receiver_account_id AS account_id,
    'FAN_IN' AS rule_name,
    COUNT(DISTINCT t2.sender_account_id) AS unique_originators_24h,
    ROUND(SUM(t2.amount_eur), 2) AS amount_24h_eur,
    MIN(t2.transaction_timestamp) AS window_start,
    MAX(t2.transaction_timestamp) AS window_end
FROM transactions t1
JOIN transactions t2
    ON t1.receiver_account_id = t2.receiver_account_id
   AND t2.transaction_timestamp BETWEEN t1.transaction_timestamp
       AND datetime(t1.transaction_timestamp, '+24 hours')
GROUP BY t1.receiver_account_id, t1.transaction_timestamp
HAVING COUNT(DISTINCT t2.sender_account_id) >= 5;

-- 3. Fan-out: one account sends funds to several unique beneficiaries in 24 hours.
SELECT
    t1.sender_account_id AS account_id,
    'FAN_OUT' AS rule_name,
    COUNT(DISTINCT t2.receiver_account_id) AS unique_beneficiaries_24h,
    ROUND(SUM(t2.amount_eur), 2) AS amount_24h_eur,
    MIN(t2.transaction_timestamp) AS window_start,
    MAX(t2.transaction_timestamp) AS window_end
FROM transactions t1
JOIN transactions t2
    ON t1.sender_account_id = t2.sender_account_id
   AND t2.transaction_timestamp BETWEEN t1.transaction_timestamp
       AND datetime(t1.transaction_timestamp, '+24 hours')
GROUP BY t1.sender_account_id, t1.transaction_timestamp
HAVING COUNT(DISTINCT t2.receiver_account_id) >= 5;

-- 4. High-risk geography: transaction touches a training watchlist country.
SELECT
    t.transaction_id,
    t.sender_account_id,
    t.receiver_account_id,
    t.amount_eur,
    s.country AS sender_country,
    r.country AS receiver_country
FROM transactions t
LEFT JOIN accounts s
    ON t.sender_account_id = s.account_id
LEFT JOIN accounts r
    ON t.receiver_account_id = r.account_id
WHERE s.country IN ('IRAN', 'MYANMAR', 'DPRK', 'SYRIA', 'YEMEN')
   OR r.country IN ('IRAN', 'MYANMAR', 'DPRK', 'SYRIA', 'YEMEN');

-- Optional advanced practice: circular flow A -> B -> C -> A.
-- This is included as a learning exercise, not as a required rule in the main pipeline.
SELECT
    t1.sender_account_id AS account_a,
    t1.receiver_account_id AS account_b,
    t2.receiver_account_id AS account_c,
    t3.receiver_account_id AS returned_to_account,
    t1.transaction_timestamp AS start_time,
    t3.transaction_timestamp AS end_time,
    ROUND(t1.amount_eur + t2.amount_eur + t3.amount_eur, 2) AS total_loop_amount
FROM transactions t1
JOIN transactions t2
    ON t1.receiver_account_id = t2.sender_account_id
   AND t2.transaction_timestamp > t1.transaction_timestamp
JOIN transactions t3
    ON t2.receiver_account_id = t3.sender_account_id
   AND t3.receiver_account_id = t1.sender_account_id
   AND t3.transaction_timestamp > t2.transaction_timestamp
WHERE t3.transaction_timestamp <= datetime(t1.transaction_timestamp, '+7 days');
```

`docs/export/build_project_pdf.py`

Relevant repository text file included for study context.

```
from __future__ import annotations

import csv
import os
import re
import subprocess
import textwrap
from dataclasses import dataclass
from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parents[2]
EXPORT_DIR = PROJECT_ROOT / "docs" / "export"
MARKDOWN_OUTPUT = EXPORT_DIR / "AML_project_full_study_pack.md"
PDF_OUTPUT = EXPORT_DIR / "AML_project_full_study_pack.pdf"

TEXT_EXTENSIONS = {
```

```
    ".md",
    ".py",
    ".sql",
    ".csv",
    ".txt",
    ".yml",
    ".yaml",
    ".json",
    ".toml",
    ".ini",
    ".cfg",
}
SPECIAL_TEXT_FILENAMES = {
    "README.md",
    "requirements.txt",
    "LICENSE",
    ".env.example",
    ".gitignore",
}
EXCLUDED_DIRS = {
    ".git",
    ".claude",
    ".tools",
    "node_modules",
    ".next",
    "__pycache__",
    ".pytest_cache",
    ".venv",
    "venv",
    "env",
    ".mypy_cache",
    ".ruff_cache",
    "dist",
    "build",
    "aml-interview-demo",
}
EXCLUDED_SUFFIXES = {
    ".png",
    ".jpg",
    ".jpeg",
    ".gif",
    ".webp",
    ".pdf",
    ".zip",
    ".parquet",
    ".db",
    ".sqlite",
    ".pkl",
    ".joblib",
    ".pyc",
}
MAX_FULL_TEXT_BYTES = 180_000
MAX_CSV_FULL_ROWS = 50
CSV_SAMPLE_ROWS = 20


@dataclass
class FileRecord:
    path: Path
    rel_path: str
    size_bytes: int
    source: str


@dataclass
class SkippedFile:
    rel_path: str
    reason: str


def rel(path: Path) -> str:
    return path.relative_to(PROJECT_ROOT).as_posix()


def should_exclude_path(path: Path) -> str | None:
    parts = set(path.relative_to(PROJECT_ROOT).parts)
    ignored_parts = parts & EXCLUDED_DIRS
    if ignored_parts:
        return f"excluded directory: {', '.join(sorted(ignored_parts))}"
    if path.suffix.lower() in EXCLUDED_SUFFIXES:
        return f"excluded binary/heavy suffix: {path.suffix.lower()}"
    if path.name == ".env":
        return "local environment file excluded to avoid exporting secrets"
    if path == MARKDOWN_OUTPUT or path == PDF_OUTPUT:
        return "generated export output"
    return None


def is_text_candidate(path: Path) -> bool:
    return path.suffix.lower() in TEXT_EXTENSIONS or path.name in SPECIAL_TEXT_FILENAMES


def git_tracked_files() -> set[str]:
    try:
        result = subprocess.run(
```

```
            ["git", "ls-files"],
            cwd=PROJECT_ROOT,
            text=True,
            capture_output=True,
            check=True,
        )
        return {line.strip().replace("\\", "/") for line in result.stdout.splitlines() if line.strip()}
    except Exception:
        return set()


def discover_files() -> tuple[list[FileRecord], list[SkippedFile]]:
    tracked = git_tracked_files()
    records: dict[str, FileRecord] = {}
    skipped: list[SkippedFile] = []

    for root, dirnames, filenames in os.walk(PROJECT_ROOT):
        root_path = Path(root)
        kept_dirnames = []
        for dirname in dirnames:
            directory_path = root_path / dirname
            directory_rel = rel(directory_path)
            if dirname in EXCLUDED_DIRS:
                skipped.append(SkippedFile(f"{directory_rel}/", f"excluded directory: {dirname}"))
            else:
                kept_dirnames.append(dirname)
        dirnames[:] = kept_dirnames

        for filename in filenames:
            path = root_path / filename
            if not path.is_file():
                continue

            rel_path = rel(path)
            exclusion_reason = should_exclude_path(path)
            if exclusion_reason:
                skipped.append(SkippedFile(rel_path, exclusion_reason))
                continue

            if not is_text_candidate(path):
                skipped.append(SkippedFile(rel_path, "not an included text type"))
                continue

            size_bytes = path.stat().st_size
            if size_bytes > MAX_FULL_TEXT_BYTES and path.suffix.lower() != ".csv":
                skipped.append(SkippedFile(rel_path, f"text file too large: {size_bytes} bytes"))
                continue

            source = "tracked" if rel_path in tracked else "local-generated"
            records[rel_path] = FileRecord(path=path, rel_path=rel_path, size_bytes=size_bytes, source=source)

    priority_prefixes = {
        "README.md": 0,
        "requirements.txt": 1,
        "src/": 2,
        "sql/": 3,
        "docs/": 4,
        "data/raw/": 5,
        "data/processed/": 6,
        "outputs/": 7,
        "experimental/": 8,
    }

    def sort_key(record: FileRecord) -> tuple[int, str]:
        for prefix, priority in priority_prefixes.items():
            if record.rel_path == prefix.rstrip("/") or record.rel_path.startswith(prefix):
                return priority, record.rel_path
        return 99, record.rel_path

    return sorted(records.values(), key=sort_key), sorted(skipped, key=lambda item: item.rel_path)


def read_text(path: Path) -> str:
    return path.read_text(encoding="utf-8", errors="replace")


def fence_language(path: Path) -> str:
    suffix = path.suffix.lower()
    return {
        ".py": "python",
        ".sql": "sql",
        ".csv": "csv",
        ".yml": "yaml",
        ".yaml": "yaml",
        ".json": "json",
        ".toml": "toml",
        ".ini": "ini",
        ".cfg": "ini",
        ".txt": "text",
        ".md": "markdown",
    }.get(suffix, "text")


def heading_anchor(text: str) -> str:
    return re.sub(r"\s+", " ", text).strip()
```

```
def tree_from_records(records: list[FileRecord]) -> str:
    paths = [record.rel_path for record in records]
    tree: dict[str, dict] = {}
    for path in paths:
        node = tree
        for part in path.split("/"):
            node = node.setdefault(part, {})

    def render(node: dict[str, dict], indent: int = 0) -> list[str]:
        lines: list[str] = []
        for name in sorted(node):
            suffix = "/" if node[name] else ""
            lines.append("  " * indent + name + suffix)
            if node[name]:
                lines.extend(render(node[name], indent + 1))
        return lines

    return "\n".join(render(tree))


def csv_profile(record: FileRecord) -> dict[str, object]:
    rows: list[list[str]] = []
    with record.path.open("r", encoding="utf-8", errors="replace", newline="") as handle:
        reader = csv.reader(handle)
        for index, row in enumerate(reader):
            rows.append(row)
            if index >= CSV_SAMPLE_ROWS:
                break

    row_count = 0
    with record.path.open("r", encoding="utf-8", errors="replace", newline="") as handle:
        reader = csv.reader(handle)
        header_seen = False
        for row in reader:
            if not header_seen:
                header_seen = True
                continue
            if row:
                row_count += 1

    headers = rows[0] if rows else []
    return {"headers": headers, "rows": rows, "row_count": row_count}


def markdown_table(headers: list[str], rows: list[list[str]]) -> str:
    if not headers:
        return "_No columns detected._"
    safe_headers = [cell.replace("|", "\\|") for cell in headers]
    table = ["| " + " | ".join(safe_headers) + " |"]
    table.append("| " + " | ".join("---" for _ in safe_headers) + " |")
    for row in rows:
        padded = row + [""] * (len(headers) - len(row))
        safe_row = [str(cell).replace("|", "\\|") for cell in padded[: len(headers)]]
        table.append("| " + " | ".join(safe_row) + " |")
    return "\n".join(table)


def load_csv_rows(path: Path) -> list[dict[str, str]]:
    if not path.exists():
        return []
    with path.open("r", encoding="utf-8", errors="replace", newline="") as handle:
        return list(csv.DictReader(handle))


def infer_data_dictionary(records: list[FileRecord]) -> str:
    descriptions = {
        "account_id": "Unique account identifier; inferred from file/code context.",
        "account_name": "Readable account/customer name after cleaning; inferred from file/code context.",
        "name": "Raw account name before cleaning.",
        "country": "Account country used for geography features.",
        "account_type": "Synthetic type/category of the account.",
        "transaction_id": "Unique transaction identifier.",
        "timestamp": "Raw transaction timestamp.",
        "transaction_timestamp": "Cleaned timestamp parsed as UTC datetime.",
        "transaction_date": "Date derived from the transaction timestamp.",
        "sender_account_id": "Account sending funds.",
        "receiver_account_id": "Account receiving funds.",
        "amount": "Raw transaction amount.",
        "amount_original": "Original transaction amount before conversion.",
        "amount_eur": "Transaction amount normalized to EUR in the simplified pipeline.",
        "currency": "Transaction currency.",
        "transaction_type": "Synthetic transaction category/type.",
        "is_laundering": "Synthetic label included in data; not used as real compliance evidence.",
        "sender_country": "Country joined from the sender account.",
        "receiver_country": "Country joined from the receiver account.",
        "high_risk_jurisdiction_flag": "1 when a transaction/account touches the training watchlist.",
        "transaction_count": "Total count of inbound and outbound transactions.",
        "transaction_count_out": "Count of outgoing transactions for the account.",
        "transaction_count_in": "Count of incoming transactions for the account.",
        "total_outbound_amount": "Total EUR sent by the account.",
        "total_inbound_amount": "Total EUR received by the account.",
        "average_transaction_amount": "Average transaction amount; inferred from generated features.",
        "avg_outbound_amount": "Average EUR amount for outgoing transactions.",
        "avg_inbound_amount": "Average EUR amount for incoming transactions.",
        "unique_counterparties": "Number of distinct counterparties connected to the account.",
```

```
        "unique_outbound_counterparties": "Number of distinct receivers for outgoing transactions.",
        "unique_inbound_counterparties": "Number of distinct senders for incoming transactions.",
        "number_of_countries": "Number of distinct counterparty countries.",
        "alert_id": "Generated identifier for an alert.",
        "rule_name": "Name of the detection rule that created the alert.",
        "severity": "Simple priority label derived from count-based thresholds.",
        "reason": "Human-readable explanation of why the alert was generated.",
        "metric_1_name": "Name of the first supporting metric.",
        "metric_1_value": "Value of the first supporting metric.",
        "metric_2_name": "Name of the second supporting metric.",
        "metric_2_value": "Value of the second supporting metric.",
        "detection_window_start": "Start of the rule detection window.",
        "detection_window_end": "End of the rule detection window.",
        "detection_date": "Date associated with the alert window end.",
    }

    lines = []
    csv_records = [record for record in records if record.path.suffix.lower() == ".csv"]
    seen: set[tuple[str, str]] = set()
    for record in csv_records:
        profile = csv_profile(record)
        headers = profile["headers"]
        if not headers:
            continue
        lines.append(f"### {record.rel_path}")
        lines.append("")
        lines.append(f"- Row count: {profile['row_count']}")
        lines.append("- Columns:")
        for column in headers:
            key = (record.rel_path, column)
            if key in seen:
                continue
            seen.add(key)
            lines.append(f"  - `{column}`: {descriptions.get(column, 'Meaning inferred from file/code context;
no richer definition is explicit in the repository.')}")
        lines.append("")
    return "\n".join(lines)


def rule_examples() -> dict[str, dict[str, str]]:
    rows = load_csv_rows(PROJECT_ROOT / "outputs" / "alerts_sample.csv")
    examples: dict[str, dict[str, str]] = {}
    for row in rows:
        examples.setdefault(row.get("rule_name", ""), row)
    return examples


def format_rule_example(rule_name: str, examples: dict[str, dict[str, str]]) -> str:
    row = examples.get(rule_name)
    if not row:
        return "No example alert found in `outputs/alerts_sample.csv`."
    return (
        f"Example alert: `{row.get('alert_id')}` for account `{row.get('account_id')}`, "
        f"severity `{row.get('severity')}`. Reason: {row.get('reason')}"
    )


def generated_sections(records: list[FileRecord], skipped: list[SkippedFile]) -> str:
    examples = rule_examples()
    return f"""# AML Transaction Monitoring Analytics — Full Study Pack

Generated from repository: `{PROJECT_ROOT}`

## 1. Project Overview

This repository is a simplified AML transaction monitoring analytics project for a junior Data/BI + AML
analytics portfolio. It uses synthetic account and transaction CSV files, cleans and standardizes them with
Python/Pandas, creates basic behavioural features, applies explainable rule-based alert detection, and produces
an alerts CSV for analyst review or dashboarding practice.

The main pipeline is:

1. `src/ingest_data.py`
2. `src/clean_data.py`
3. `src/generate_features.py`
4. `src/detect_alerts.py`

The main output is `outputs/alerts_sample.csv`. The project is educational, uses synthetic data, and does not
make real compliance decisions.

Core project = simplified junior analytics project. `experimental/` = optional, archived or future-work
material.

## 2. Repository Map

The following tree includes files selected for this study pack:
```

{tree_from_records(records)}

```
## 3. Execution Flow

### Step 1: `src/ingest_data.py`

- Input files: `data/raw/accounts.csv`, `data/raw/transactions.csv`.
```

- Output files: `data/processed/accounts_ingested.csv`, `data/processed/transactions_ingested.csv`.
- Main transformations: standardizes column names and adds an ingestion timestamp.
- Why it matters: creates a reproducible first landing layer while keeping the project CSV/Pandas based.

Step 2: `src/clean_data.py`

- Input files: ingested account and transaction CSVs from `data/processed/`.
- Output files: `data/processed/accounts_clean.csv`, `data/processed/transactions_clean.csv`.
- Main transformations: renames key fields, parses timestamps, normalizes currencies, converts amounts to EUR, handles missing values, filters invalid rows and removes duplicates.
- Why it matters: creates consistent data for reliable features and rules.

Step 3: `src/generate_features.py`

- Input files: clean account and transaction CSVs.
- Output files: `data/processed/account_features.csv`, `data/processed/transaction_features.csv`.
- Main transformations: joins country context, calculates inbound/outbound totals, transaction counts, unique counterparties, country counts, average amount and a high-risk jurisdiction flag.
- Why it matters: turns raw transactions into analyst-friendly behavioural indicators.

Step 4: `src/detect_alerts.py`

- Input file: `data/processed/transaction_features.csv`.
- Output file: `outputs/alerts_sample.csv`.
- Main transformations: applies smurfing, fan-in, fan-out and high-risk geography rules.
- Why it matters: produces a final table that can be reviewed by an analyst or used in a dashboard.

4. Data Dictionary

The definitions below are generated from CSV headers and code context. When the repository does not define a business meaning explicitly, the description says it is inferred.

{infer_data_dictionary(records)}

5. Detection Rules Explained

Smurfing / Structuring

- Business intuition: repeated smaller outgoing transfers may indicate an attempt to split value into multiple payments.
- Technical logic: `detect_smurfing` checks outgoing transactions below EUR 9,999 and looks for at least 5 within a 72-hour window.
- Input columns: `sender_account_id`, `amount_eur`, `transaction_timestamp`.
- Output columns: `alert_id`, `account_id`, `rule_name`, `severity`, `reason`, supporting metric fields and detection window fields.
- Limitations: fixed thresholds, no customer profile, no historical baseline and no real compliance decision.
- {format_rule_example("SMURFING", examples)}

Fan-In

- Business intuition: one account receiving funds from many unique originators in a short window may indicate collection-account behaviour.
- Technical logic: `detect_fan_pattern` groups by `receiver_account_id` and counts unique `sender_account_id` values within 24 hours.
- Input columns: `receiver_account_id`, `sender_account_id`, `amount_eur`, `transaction_timestamp`.
- Output columns: same alert schema as other rules.
- Limitations: can be normal for some business accounts; requires profile and activity-context review.
- {format_rule_example("FAN_IN", examples)}

Fan-Out

- Business intuition: one account sending funds to many unique beneficiaries in a short window may indicate dispersion behaviour.
- Technical logic: `detect_fan_pattern` groups by `sender_account_id` and counts unique `receiver_account_id` values within 24 hours.
- Input columns: `sender_account_id`, `receiver_account_id`, `amount_eur`, `transaction_timestamp`.
- Output columns: same alert schema as other rules.
- Limitations: can be normal for payroll, refunds or supplier payments; further analyst review is required.
- {format_rule_example("FAN_OUT", examples)}

High-Risk Geography

- Business intuition: activity involving a watchlist country may require extra context.
- Technical logic: `detect_high_risk_geography` checks whether sender or receiver country appears in the training watchlist: `IRAN`, `MYANMAR`, `DPRK`, `SYRIA`, `YEMEN`.
- Input columns: `sender_country`, `receiver_country`, `amount_eur`, account identifiers and timestamp.
- Output columns: same alert schema as other rules.
- Limitations: the country list is hardcoded for training and is not a maintained regulatory source.
- {format_rule_example("HIGH_RISK_GEOGRAPHY", examples)}

Circular Flow

- Business intuition: A -> B -> C -> A loops can indicate circular movement of funds.
- Technical logic: present as optional SQL practice in `sql/03_detection_rules.sql`, not as part of the core Python pipeline.
- Input columns: sender/receiver account IDs, transaction timestamps and EUR amount.
- Output columns: query result fields such as account path, time window and total loop amount.
- Limitations: marked as advanced practice; not a core junior-project detection rule.

6. Main Documentation Files
"""

def file_explanation(record: FileRecord) -> str:
    path = record.rel_path

```
    explanations = {
        "README.md": "Main project description, positioning, run instructions and scope.",
        "requirements.txt": "Minimal Python dependencies for the simplified pipeline.",
        "src/ingest_data.py": "Loads raw CSV files, standardizes headers and writes the ingested layer.",
        "src/clean_data.py": "Cleans accounts and transactions, parses timestamps and normalizes amounts.",
        "src/generate_features.py": "Builds transaction-level and account-level analytical features.",
        "src/detect_alerts.py": "Applies simplified rule-based AML alert logic and writes the alerts output.",
        "sql/01_create_tables.sql": "Creates simple SQL tables for accounts and transactions.",
        "sql/02_account_features.sql": "Shows SQL logic for account-level features.",
        "sql/03_detection_rules.sql": "Shows SQL examples for simplified detection rules and optional circular
flow.",
        "docs/methodology.md": "Explains the project methodology in study-friendly language.",
        "docs/case_studies.md": "Provides realistic junior analyst case narratives.",
        "outputs/alerts_sample.csv": "Final sample alerts table generated by the rule-based pipeline.",
    }
    if path.startswith("data/raw/"):
        return "Small synthetic raw input CSV used by the simplified project."
    if path.startswith("data/processed/"):
        return "Generated processed CSV from the local pipeline run; useful as a study artifact."
    if path.startswith("experimental/"):
        return "Optional/non-core file retained for archived advanced work or future improvements."
    return explanations.get(path, "Relevant repository text file included for study context.")


def append_full_file_section(lines: list[str], record: FileRecord, level: int = 3) -> None:
    hashes = "#" * level
    lines.append(f"{hashes} `{record.rel_path}`")
    lines.append("")
    lines.append(file_explanation(record))
    lines.append("")

    if record.path.suffix.lower() == ".csv":
        profile = csv_profile(record)
        rows = profile["rows"]
        headers = profile["headers"]
        data_rows = rows[1:]
        lines.append(f"- Row count: {profile['row_count']}")
        lines.append(f"- Columns: {', '.join(f'`{column}`' for column in headers) if headers else 'none
detected'}")
        lines.append("")
        if profile["row_count"] <= MAX_CSV_FULL_ROWS:
            lines.append("Full CSV content:")
            lines.append("")
            lines.append(f"```csv\n{read_text(record.path).strip()}\n```")
        else:
            lines.append(f"First {CSV_SAMPLE_ROWS} rows:")
            lines.append("")
            lines.append(markdown_table(headers, data_rows[:CSV_SAMPLE_ROWS]))
        lines.append("")
        return

    language = fence_language(record.path)
    content = read_text(record.path).rstrip()
    lines.append(f"```{language}")
    lines.append(content)
    lines.append("```")
    lines.append("")


def build_markdown(records: list[FileRecord], skipped: list[SkippedFile]) -> str:
    lines: list[str] = []
    lines.append(generated_sections(records, skipped))

    doc_records = [
        record
        for record in records
        if record.rel_path == "README.md"
        or (record.rel_path.startswith("docs/") and record.path.suffix.lower() == ".md" and record.rel_path !=
"docs/export/AML_project_full_study_pack.md")
    ]
    for record in doc_records:
        append_full_file_section(lines, record)

    lines.append("## 7. Source Code")
    lines.append("")
    code_records = [
        record
        for record in records
        if record.rel_path.startswith("src/")
        or record.rel_path.startswith("sql/")
        or record.rel_path == "docs/export/build_project_pdf.py"
        or record.rel_path in {"requirements.txt", ".gitignore"}
    ]
    for record in code_records:
        append_full_file_section(lines, record)

    lines.append("## 8. Data Samples")
    lines.append("")
    data_records = [
        record
        for record in records
        if record.rel_path.startswith("data/raw/")
        or record.rel_path.startswith("data/processed/")
        or record.rel_path.startswith("outputs/")
    ]
```

```
    for record in data_records:
        append_full_file_section(lines, record)

    lines.append("## 9. Experimental / Archived Content")
    lines.append("")
    lines.append("The files below are optional/non-core. They preserve previous or advanced work, but the main
interview story should stay focused on the simplified CSV/Pandas/SQL rule-based pipeline.")
    lines.append("")
    experimental_records = [record for record in records if record.rel_path.startswith("experimental/")]
    for record in experimental_records:
        append_full_file_section(lines, record, level=3)

    lines.append("## 10. Interview Defense Notes")
    lines.append("")
    lines.append("### 60-second explanation")
    lines.append("")
    lines.append("This is a simplified AML analytics portfolio project using synthetic data. It loads account
and transaction CSVs, cleans them with Python/Pandas, creates behavioural features, applies a small set of
explainable rules, and outputs an alerts table for analyst review or dashboarding. It is not a compliance
decision system; it is a practical project to demonstrate data cleaning, feature engineering, SQL thinking and
AML monitoring concepts at junior level.")
    lines.append("")
    lines.append("### Technical decisions")
    lines.append("")
    lines.append("- Kept the main path CSV/Pandas based so it is easy to run and explain.")
    lines.append("- Used explainable rules instead of ML as the core detection approach.")
    lines.append("- Generated both account-level and transaction-level features.")
    lines.append("- Kept SQL examples for interview discussion and BI/database practice.")
    lines.append("- Archived advanced work under `experimental/` instead of deleting it.")
    lines.append("")
    lines.append("### Trade-offs")
    lines.append("")
    lines.append("- Simplicity over orchestration: easier to understand, but not scheduled.")
    lines.append("- Fixed thresholds over adaptive models: explainable, but less flexible.")
    lines.append("- Synthetic data over real data: safe for a portfolio, but less realistic.")
    lines.append("- Rule-based alerts over final decisions: appropriate for education, but limited for real
compliance.")
    lines.append("")
    lines.append("### Why the project was simplified")
    lines.append("")
    lines.append("The simplified version is better aligned with junior Data Analyst, BI Analyst and AML
Analytics Junior roles. It shows the author understands the data workflow and AML monitoring logic without
overstating production readiness.")
    lines.append("")
    lines.append("### Limitations")
    lines.append("")
    lines.append("- No real customer due diligence data.")
    lines.append("- No analyst feedback loop.")
    lines.append("- No live regulatory watchlist refresh.")
    lines.append("- No transaction monitoring calibration process.")
    lines.append("- No production controls, access management or audit workflow.")
    lines.append("")
    lines.append("### Future improvements")
    lines.append("")
    lines.append("- Add a small Streamlit or Power BI dashboard.")
    lines.append("- Add configurable rule thresholds.")
    lines.append("- Add analyst review status fields.")
    lines.append("- Move the SQL examples into SQLite/PostgreSQL exercises.")
    lines.append("- Revisit `experimental/` for orchestration, graph analytics or ML only after the core story
is solid.")
    lines.append("")
    lines.append("### Likely interview questions and strong answers")
    lines.append("")
    lines.append("**Q: Is this a real AML system?**  ")
    lines.append("A: No. It is an educational portfolio project using synthetic data. It generates explainable
alerts for practice, but it does not make compliance decisions.")
    lines.append("")
    lines.append("**Q: Why rule-based detection instead of ML?**  ")
    lines.append("A: For junior analytics roles, rule-based detection is easier to explain, audit and connect to
 AML typologies. ML can be future work, but the main learning goal is clean data and transparent alert logic.")
    lines.append("")
    lines.append("**Q: What would you improve first?**  ")
    lines.append("A: I would add configurable thresholds, a small dashboard and an analyst feedback field so
reviewed alerts can be tracked.")
    lines.append("")
    lines.append("**Q: What does `severity` mean here?**  ")
    lines.append("A: It is a simple priority label based on rule metrics, not a final risk rating. It helps
order alerts for review.")
    lines.append("")
    lines.append("**Q: How would this differ in a real institution?**  ")
    lines.append("A: A real institution would require governance, threshold calibration, KYC context, alert
investigation workflow, audit trails, model/rule validation and regulatory procedures.")
    lines.append("")
    lines.append("## 11. Glossary")
    lines.append("")
    glossary = {
        "AML": "Anti-Money Laundering; controls and analysis used to identify and investigate suspicious
financial activity.",
        "Transaction monitoring": "Review of transaction activity to detect unusual or potentially suspicious
patterns.",
        "Alert": "A record generated by a rule or model that requires review.",
        "Smurfing": "Splitting funds into multiple smaller transactions, often discussed as structuring.",
        "Fan-in": "Many senders transferring into one receiver within a short period.",
        "Fan-out": "One sender transferring to many receivers within a short period.",
        "High-risk geography": "Activity involving a country or jurisdiction that requires additional context or
```

```
 review.",
        "Synthetic data": "Artificial data created for learning or testing, not real customer data.",
        "Feature engineering": "Creating analytical variables from raw data, such as counts, totals or flags.",
        "Severity": "A simple priority label for an alert in this project.",
        "False positive": "An alert that looks unusual by rule logic but is later explained as legitimate.",
        "Analyst review": "Human investigation step where context, customer profile and evidence are assessed.",
    }
    for term, definition in glossary.items():
        lines.append(f"- **{term}:** {definition}")
    lines.append("")

    lines.append("## Skipped Files")
    lines.append("")
    lines.append("The following files were skipped to avoid heavy, binary, irrelevant or sensitive content.")
    lines.append("")
    for item in skipped:
        lines.append(f"- `{item.rel_path}`: {item.reason}")
    lines.append("")
    return "\n".join(lines)


def markdown_to_pdf(markdown_text: str) -> None:
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import cm
    from reportlab.platypus import PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer

    styles = getSampleStyleSheet()
    normal = ParagraphStyle("StudyNormal", parent=styles["BodyText"], fontName="Helvetica", fontSize=9,
leading=12)
    code = ParagraphStyle("StudyCode", parent=styles["Code"], fontName="Courier", fontSize=6.5, leading=8,
textColor=colors.HexColor("#222222"))
    headings = {
        1: ParagraphStyle("H1", parent=styles["Heading1"], fontSize=18, leading=22, spaceAfter=10),
        2: ParagraphStyle("H2", parent=styles["Heading2"], fontSize=14, leading=18, spaceBefore=8,
spaceAfter=6),
        3: ParagraphStyle("H3", parent=styles["Heading3"], fontSize=11, leading=14, spaceBefore=6,
spaceAfter=4),
    }

    doc = SimpleDocTemplate(
        str(PDF_OUTPUT),
        pagesize=A4,
        leftMargin=1.3 * cm,
        rightMargin=1.3 * cm,
        topMargin=1.2 * cm,
        bottomMargin=1.2 * cm,
        title="AML Transaction Monitoring Analytics Full Study Pack",
    )

    story = []
    in_code = False
    code_lines: list[str] = []
    paragraph_lines: list[str] = []

    def flush_paragraph() -> None:
        if not paragraph_lines:
            return
        text = " ".join(paragraph_lines).strip()
        paragraph_lines.clear()
        if not text:
            return
        escaped = (
            text.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("**", "")
            .replace("`", "")
        )
        story.append(Paragraph(escaped, normal))
        story.append(Spacer(1, 4))

    def flush_code() -> None:
        if not code_lines:
            return
        chunk = "\n".join(code_lines)
        code_lines.clear()
        wrapped_lines = []
        for line in chunk.splitlines():
            wrapped_lines.extend(textwrap.wrap(line, width=112, replace_whitespace=False, drop_whitespace=False)
 or [""])
        story.append(Preformatted("\n".join(wrapped_lines), code))
        story.append(Spacer(1, 6))

    for raw_line in markdown_text.splitlines():
        line = raw_line.rstrip("\n")
        if line.startswith("```"):
            if in_code:
                flush_code()
                in_code = False
            else:
                flush_paragraph()
                in_code = True
            continue
```

```
        if in_code:
            code_lines.append(line)
            continue

        if not line.strip():
            flush_paragraph()
            continue

        heading_match = re.match(r"^(#{1,3})\s+(.*)$", line)
        if heading_match:
            flush_paragraph()
            level = len(heading_match.group(1))
            text = heading_anchor(heading_match.group(2)).replace("&", "&amp;").replace("<",
"&lt;").replace(">", "&gt;")
            if level == 1 and story:
                story.append(PageBreak())
            story.append(Paragraph(text, headings[level]))
            continue

        if line.startswith("|") or line.startswith("- ") or re.match(r"^\d+\.\s", line):
            flush_paragraph()
            clean = line.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;").replace("`", "")
            story.append(Paragraph(clean, normal))
            story.append(Spacer(1, 2))
            continue

        paragraph_lines.append(line)

    flush_paragraph()
    flush_code()
    doc.build(story)


def main() -> None:
    EXPORT_DIR.mkdir(parents=True, exist_ok=True)
    records, skipped = discover_files()
    markdown_text = build_markdown(records, skipped)
    MARKDOWN_OUTPUT.write_text(markdown_text, encoding="utf-8")
    markdown_to_pdf(markdown_text)

    print(f"Markdown written to: {MARKDOWN_OUTPUT}")
    print(f"PDF written to: {PDF_OUTPUT}")
    print(f"Included files: {len(records)}")
    print(f"Skipped files: {len(skipped)}")


if __name__ == "__main__":
    main()
```

`.gitignore`

Relevant repository text file included for study context.

```
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.venv/
venv/
dist/
build/
logs/

data/*
!data/.gitkeep
!data/raw/
!data/raw/*.csv
!data/processed/
data/processed/*
!data/processed/.gitkeep

outputs/*
!outputs/.gitkeep
!outputs/alerts_sample.csv

dbt/target/
dbt/logs/
dbt/packages/
experimental/orchestration/dbt/target/
experimental/orchestration/dbt/logs/
experimental/orchestration/dbt/packages/

.env
.env.*
!.env.example

.env.local
```

8. Data Samples

`data/raw/accounts.csv`

Small synthetic raw input CSV used by the simplified project.

- Row count: 25

- Columns: account_id, name, country, account_type

Full CSV content:

```
account_id,name,country,account_type
ACC-ORIGIN,Origin Customer,SPAIN,INDIVIDUAL
ACC-SMURF,North Market Wallet,SPAIN,INDIVIDUAL
ACC-BEN-01,Beneficiary One,SPAIN,INDIVIDUAL
ACC-BEN-02,Beneficiary Two,PORTUGAL,INDIVIDUAL
ACC-BEN-03,Beneficiary Three,FRANCE,INDIVIDUAL
ACC-BEN-04,Beneficiary Four,GERMANY,INDIVIDUAL
ACC-BEN-05,Beneficiary Five,ITALY,INDIVIDUAL
ACC-FANIN,Iberia Collection Account,SPAIN,BUSINESS
ACC-SEND-01,Sender One,FRANCE,INDIVIDUAL
ACC-SEND-02,Sender Two,GERMANY,INDIVIDUAL
ACC-SEND-03,Sender Three,ITALY,INDIVIDUAL
ACC-SEND-04,Sender Four,PORTUGAL,INDIVIDUAL
ACC-SEND-05,Sender Five,NETHERLANDS,INDIVIDUAL
ACC-FANOUT,Distribution Services,SPAIN,BUSINESS
ACC-REC-01,Receiver One,SPAIN,INDIVIDUAL
ACC-REC-02,Receiver Two,PORTUGAL,INDIVIDUAL
ACC-REC-03,Receiver Three,FRANCE,INDIVIDUAL
ACC-REC-04,Receiver Four,GERMANY,INDIVIDUAL
ACC-REC-05,Receiver Five,ITALY,INDIVIDUAL
ACC-GEO,Travel Import Trader,SPAIN,BUSINESS
ACC-HR-01,High Risk Counterparty One,IRAN,BUSINESS
ACC-HR-02,High Risk Counterparty Two,MYANMAR,BUSINESS
ACC-CIRC-A,Circular Account A,SPAIN,BUSINESS
ACC-CIRC-B,Circular Account B,PORTUGAL,BUSINESS
ACC-CIRC-C,Circular Account C,FRANCE,BUSINESS
```

`data/raw/transactions.csv`

Small synthetic raw input CSV used by the simplified project.

- Row count: 21

- Columns: transaction_id, timestamp, sender_account_id, receiver_account_id, amount, currency, transaction_type, is_laundering

Full CSV content:

```
transaction_id,timestamp,sender_account_id,receiver_account_id,amount,currency,transaction_type,is_laundering
TX-0001,2024-01-02T09:15:00Z,ACC-ORIGIN,ACC-SMURF,52000,EUR,WIRE_IN,0
TX-0002,2024-01-02T12:10:00Z,ACC-SMURF,ACC-BEN-01,9800,EUR,TRANSFER,0
TX-0003,2024-01-02T15:20:00Z,ACC-SMURF,ACC-BEN-02,9500,EUR,TRANSFER,0
TX-0004,2024-01-03T09:40:00Z,ACC-SMURF,ACC-BEN-03,8700,EUR,TRANSFER,0
TX-0005,2024-01-03T13:35:00Z,ACC-SMURF,ACC-BEN-04,9100,EUR,TRANSFER,0
TX-0006,2024-01-04T08:05:00Z,ACC-SMURF,ACC-BEN-05,8900,EUR,TRANSFER,0
TX-0007,2024-01-08T10:00:00Z,ACC-SEND-01,ACC-FANIN,4100,EUR,TRANSFER,0
TX-0008,2024-01-08T11:10:00Z,ACC-SEND-02,ACC-FANIN,3900,EUR,TRANSFER,0
TX-0009,2024-01-08T12:25:00Z,ACC-SEND-03,ACC-FANIN,4200,EUR,TRANSFER,0
TX-0010,2024-01-08T14:00:00Z,ACC-SEND-04,ACC-FANIN,4050,EUR,TRANSFER,0
TX-0011,2024-01-08T16:30:00Z,ACC-SEND-05,ACC-FANIN,3950,EUR,TRANSFER,0
TX-0012,2024-01-10T09:00:00Z,ACC-FANOUT,ACC-REC-01,10300,EUR,TRANSFER,0
TX-0013,2024-01-10T10:15:00Z,ACC-FANOUT,ACC-REC-02,10550,EUR,TRANSFER,0
TX-0014,2024-01-10T11:45:00Z,ACC-FANOUT,ACC-REC-03,10700,EUR,TRANSFER,0
TX-0015,2024-01-10T13:20:00Z,ACC-FANOUT,ACC-REC-04,10400,EUR,TRANSFER,0
TX-0016,2024-01-10T15:55:00Z,ACC-FANOUT,ACC-REC-05,10650,EUR,TRANSFER,0
TX-0017,2024-01-12T09:30:00Z,ACC-HR-01,ACC-GEO,12500,EUR,TRANSFER,0
TX-0018,2024-01-12T15:45:00Z,ACC-GEO,ACC-HR-02,11800,EUR,TRANSFER,0
TX-0019,2024-01-15T10:00:00Z,ACC-CIRC-A,ACC-CIRC-B,15000,EUR,TRANSFER,0
TX-0020,2024-01-16T10:00:00Z,ACC-CIRC-B,ACC-CIRC-C,14500,EUR,TRANSFER,0
TX-0021,2024-01-17T10:00:00Z,ACC-CIRC-C,ACC-CIRC-A,14000,EUR,TRANSFER,0
```

`data/processed/account_features.csv`

Generated processed CSV from the local pipeline run; useful as a study artifact.

- Row count: 25

- Columns: account_id, account_name, country, account_type, transaction_count_out, total_outbound_amount, avg_outbound_amount, unique_outbound_counterparties, transaction_count_in, total_inbound_amount, avg_inbound_amount, unique_inbound_counterparties, unique_counterparties, number_of_countries, high_risk_jurisdiction_flag, transaction_count, average_transaction_amount

Full CSV content:

```
account_id,account_name,country,account_type,transaction_count_out,total_outbound_amount,avg_outbound_amount,uni
que_outbound_counterparties,transaction_count_in,total_inbound_amount,avg_inbound_amount,unique_inbound_counterp
arties,unique_counterparties,number_of_countries,high_risk_jurisdiction_flag,transaction_count,average_transacti
```

```
on_amount
ACC-ORIGIN,Origin Customer,SPAIN,INDIVIDUAL,1.0,52000.0,52000.0,1.0,0.0,0.0,0.0,0.0,1,1,0,1,52000.0
ACC-SMURF,North Market Wallet,SPAIN,INDIVIDUAL,5.0,46000.0,9200.0,5.0,1.0,52000.0,52000.0,1.0,6,5,0,6,16333.33
ACC-BEN-01,Beneficiary One,SPAIN,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,9800.0,9800.0,1.0,1,1,0,1,9800.0
ACC-BEN-02,Beneficiary Two,PORTUGAL,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,9500.0,9500.0,1.0,1,1,0,1,9500.0
ACC-BEN-03,Beneficiary Three,FRANCE,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,8700.0,8700.0,1.0,1,1,0,1,8700.0
ACC-BEN-04,Beneficiary Four,GERMANY,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,9100.0,9100.0,1.0,1,1,0,1,9100.0
ACC-BEN-05,Beneficiary Five,ITALY,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,8900.0,8900.0,1.0,1,1,0,1,8900.0
ACC-FANIN,Iberia Collection Account,SPAIN,BUSINESS,0.0,0.0,0.0,0.0,0.0,5.0,20200.0,4040.0,5.0,5,5,0,5,4040.0
ACC-SEND-01,Sender One,FRANCE,INDIVIDUAL,1.0,4100.0,4100.0,1.0,0.0,0.0,0.0,0.0,1,1,0,1,4100.0
ACC-SEND-02,Sender Two,GERMANY,INDIVIDUAL,1.0,3900.0,3900.0,1.0,0.0,0.0,0.0,0.0,1,1,0,1,3900.0
ACC-SEND-03,Sender Three,ITALY,INDIVIDUAL,1.0,4200.0,4200.0,1.0,0.0,0.0,0.0,0.0,1,1,0,1,4200.0
ACC-SEND-04,Sender Four,PORTUGAL,INDIVIDUAL,1.0,4050.0,4050.0,1.0,0.0,0.0,0.0,0.0,1,1,0,1,4050.0
ACC-SEND-05,Sender Five,NETHERLANDS,INDIVIDUAL,1.0,3950.0,3950.0,1.0,0.0,0.0,0.0,0.0,1,1,0,1,3950.0
ACC-FANOUT,Distribution Services,SPAIN,BUSINESS,5.0,52600.0,10520.0,5.0,0.0,0.0,0.0,0.0,5,5,0,5,10520.0
ACC-REC-01,Receiver One,SPAIN,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,10300.0,10300.0,1.0,1,1,0,1,10300.0
ACC-REC-02,Receiver Two,PORTUGAL,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,10550.0,10550.0,1.0,1,1,0,1,10550.0
ACC-REC-03,Receiver Three,FRANCE,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,10700.0,10700.0,1.0,1,1,0,1,10700.0
ACC-REC-04,Receiver Four,GERMANY,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,10400.0,10400.0,1.0,1,1,0,1,10400.0
ACC-REC-05,Receiver Five,ITALY,INDIVIDUAL,0.0,0.0,0.0,0.0,1.0,10650.0,10650.0,1.0,1,1,0,1,10650.0
ACC-GEO,Travel Import Trader,SPAIN,BUSINESS,1.0,11800.0,11800.0,1.0,1.0,12500.0,12500.0,1.0,2,2,1,2,12150.0
ACC-HR-01,High Risk Counterparty One,IRAN,BUSINESS,1.0,12500.0,12500.0,1.0,0.0,0.0,0.0,0.0,1,1,1,1,12500.0
ACC-HR-02,High Risk Counterparty Two,MYANMAR,BUSINESS,0.0,0.0,0.0,0.0,1.0,11800.0,11800.0,1.0,1,1,1,1,11800.0
ACC-CIRC-A,Circular Account A,SPAIN,BUSINESS,1.0,15000.0,15000.0,1.0,1.0,14000.0,14000.0,1.0,2,2,0,2,14500.0
ACC-CIRC-B,Circular Account B,PORTUGAL,BUSINESS,1.0,14500.0,14500.0,1.0,1.0,15000.0,15000.0,1.0,2,2,0,2,14750.0
ACC-CIRC-C,Circular Account C,FRANCE,BUSINESS,1.0,14000.0,14000.0,1.0,1.0,14500.0,14500.0,1.0,2,2,0,2,14250.0
```

`data/processed/accounts_clean.csv`

Generated processed CSV from the local pipeline run; useful as a study artifact.

- Row count: 25

- Columns: account_id, account_name, country, account_type

Full CSV content:

```
account_id,account_name,country,account_type
ACC-ORIGIN,Origin Customer,SPAIN,INDIVIDUAL
ACC-SMURF,North Market Wallet,SPAIN,INDIVIDUAL
ACC-BEN-01,Beneficiary One,SPAIN,INDIVIDUAL
ACC-BEN-02,Beneficiary Two,PORTUGAL,INDIVIDUAL
ACC-BEN-03,Beneficiary Three,FRANCE,INDIVIDUAL
ACC-BEN-04,Beneficiary Four,GERMANY,INDIVIDUAL
ACC-BEN-05,Beneficiary Five,ITALY,INDIVIDUAL
ACC-FANIN,Iberia Collection Account,SPAIN,BUSINESS
ACC-SEND-01,Sender One,FRANCE,INDIVIDUAL
ACC-SEND-02,Sender Two,GERMANY,INDIVIDUAL
ACC-SEND-03,Sender Three,ITALY,INDIVIDUAL
ACC-SEND-04,Sender Four,PORTUGAL,INDIVIDUAL
ACC-SEND-05,Sender Five,NETHERLANDS,INDIVIDUAL
ACC-FANOUT,Distribution Services,SPAIN,BUSINESS
ACC-REC-01,Receiver One,SPAIN,INDIVIDUAL
ACC-REC-02,Receiver Two,PORTUGAL,INDIVIDUAL
ACC-REC-03,Receiver Three,FRANCE,INDIVIDUAL
ACC-REC-04,Receiver Four,GERMANY,INDIVIDUAL
ACC-REC-05,Receiver Five,ITALY,INDIVIDUAL
ACC-GEO,Travel Import Trader,SPAIN,BUSINESS
ACC-HR-01,High Risk Counterparty One,IRAN,BUSINESS
ACC-HR-02,High Risk Counterparty Two,MYANMAR,BUSINESS
ACC-CIRC-A,Circular Account A,SPAIN,BUSINESS
ACC-CIRC-B,Circular Account B,PORTUGAL,BUSINESS
ACC-CIRC-C,Circular Account C,FRANCE,BUSINESS
```

`data/processed/accounts_ingested.csv`

Generated processed CSV from the local pipeline run; useful as a study artifact.

- Row count: 25

- Columns: account_id, name, country, account_type, ingested_at

Full CSV content:

```
account_id,name,country,account_type,ingested_at
ACC-ORIGIN,Origin Customer,SPAIN,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-SMURF,North Market Wallet,SPAIN,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-BEN-01,Beneficiary One,SPAIN,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-BEN-02,Beneficiary Two,PORTUGAL,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-BEN-03,Beneficiary Three,FRANCE,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-BEN-04,Beneficiary Four,GERMANY,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-BEN-05,Beneficiary Five,ITALY,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-FANIN,Iberia Collection Account,SPAIN,BUSINESS,2026-04-26T14:37:11.041701+00:00
ACC-SEND-01,Sender One,FRANCE,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-SEND-02,Sender Two,GERMANY,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-SEND-03,Sender Three,ITALY,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-SEND-04,Sender Four,PORTUGAL,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-SEND-05,Sender Five,NETHERLANDS,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-FANOUT,Distribution Services,SPAIN,BUSINESS,2026-04-26T14:37:11.041701+00:00
```

```
ACC-REC-01,Receiver One,SPAIN,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-REC-02,Receiver Two,PORTUGAL,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-REC-03,Receiver Three,FRANCE,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-REC-04,Receiver Four,GERMANY,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-REC-05,Receiver Five,ITALY,INDIVIDUAL,2026-04-26T14:37:11.041701+00:00
ACC-GEO,Travel Import Trader,SPAIN,BUSINESS,2026-04-26T14:37:11.041701+00:00
ACC-HR-01,High Risk Counterparty One,IRAN,BUSINESS,2026-04-26T14:37:11.041701+00:00
ACC-HR-02,High Risk Counterparty Two,MYANMAR,BUSINESS,2026-04-26T14:37:11.041701+00:00
ACC-CIRC-A,Circular Account A,SPAIN,BUSINESS,2026-04-26T14:37:11.041701+00:00
ACC-CIRC-B,Circular Account B,PORTUGAL,BUSINESS,2026-04-26T14:37:11.041701+00:00
ACC-CIRC-C,Circular Account C,FRANCE,BUSINESS,2026-04-26T14:37:11.041701+00:00
```

`data/processed/transaction_features.csv`

Generated processed CSV from the local pipeline run; useful as a study artifact.

- Row count: 21

- Columns: transaction_id, transaction_timestamp, transaction_date, sender_account_id, receiver_account_id, amount_original, amount_eur, currency, transaction_type, is_laundering, sender_country, receiver_country, high_risk_jurisdiction_flag

Full CSV content:

```
transaction_id,transaction_timestamp,transaction_date,sender_account_id,receiver_account_id,amount_original,amou
nt_eur,currency,transaction_type,is_laundering,sender_country,receiver_country,high_risk_jurisdiction_flag
TX-0001,2024-01-02 09:15:00+00:00,2024-01-02,ACC-ORIGIN,ACC-SMURF,52000,52000.0,EUR,WIRE_IN,0,SPAIN,SPAIN,0
TX-0002,2024-01-02 12:10:00+00:00,2024-01-02,ACC-SMURF,ACC-BEN-01,9800,9800.0,EUR,TRANSFER,0,SPAIN,SPAIN,0
TX-0003,2024-01-02 15:20:00+00:00,2024-01-02,ACC-SMURF,ACC-BEN-02,9500,9500.0,EUR,TRANSFER,0,SPAIN,PORTUGAL,0
TX-0004,2024-01-03 09:40:00+00:00,2024-01-03,ACC-SMURF,ACC-BEN-03,8700,8700.0,EUR,TRANSFER,0,SPAIN,FRANCE,0
TX-0005,2024-01-03 13:35:00+00:00,2024-01-03,ACC-SMURF,ACC-BEN-04,9100,9100.0,EUR,TRANSFER,0,SPAIN,GERMANY,0
TX-0006,2024-01-04 08:05:00+00:00,2024-01-04,ACC-SMURF,ACC-BEN-05,8900,8900.0,EUR,TRANSFER,0,SPAIN,ITALY,0
TX-0007,2024-01-08 10:00:00+00:00,2024-01-08,ACC-SEND-01,ACC-FANIN,4100,4100.0,EUR,TRANSFER,0,FRANCE,SPAIN,0
TX-0008,2024-01-08 11:10:00+00:00,2024-01-08,ACC-SEND-02,ACC-FANIN,3900,3900.0,EUR,TRANSFER,0,GERMANY,SPAIN,0
TX-0009,2024-01-08 12:25:00+00:00,2024-01-08,ACC-SEND-03,ACC-FANIN,4200,4200.0,EUR,TRANSFER,0,ITALY,SPAIN,0
TX-0010,2024-01-08 14:00:00+00:00,2024-01-08,ACC-SEND-04,ACC-FANIN,4050,4050.0,EUR,TRANSFER,0,PORTUGAL,SPAIN,0
TX-0011,2024-01-08 16:30:00+00:00,2024-01-08,ACC-SEND-05,ACC-
FANIN,3950,3950.0,EUR,TRANSFER,0,NETHERLANDS,SPAIN,0
TX-0012,2024-01-10 09:00:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-01,10300,10300.0,EUR,TRANSFER,0,SPAIN,SPAIN,0
TX-0013,2024-01-10 10:15:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-02,10550,10550.0,EUR,TRANSFER,0,SPAIN,PORTUGAL,0
TX-0014,2024-01-10 11:45:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-03,10700,10700.0,EUR,TRANSFER,0,SPAIN,FRANCE,0
TX-0015,2024-01-10 13:20:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-04,10400,10400.0,EUR,TRANSFER,0,SPAIN,GERMANY,0
TX-0016,2024-01-10 15:55:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-05,10650,10650.0,EUR,TRANSFER,0,SPAIN,ITALY,0
TX-0017,2024-01-12 09:30:00+00:00,2024-01-12,ACC-HR-01,ACC-GEO,12500,12500.0,EUR,TRANSFER,0,IRAN,SPAIN,1
TX-0018,2024-01-12 15:45:00+00:00,2024-01-12,ACC-GEO,ACC-HR-02,11800,11800.0,EUR,TRANSFER,0,SPAIN,MYANMAR,1
TX-0019,2024-01-15 10:00:00+00:00,2024-01-15,ACC-CIRC-A,ACC-CIRC-B,15000,15000.0,EUR,TRANSFER,0,SPAIN,PORTUGAL,0
TX-0020,2024-01-16 10:00:00+00:00,2024-01-16,ACC-CIRC-B,ACC-
CIRC-C,14500,14500.0,EUR,TRANSFER,0,PORTUGAL,FRANCE,0
TX-0021,2024-01-17 10:00:00+00:00,2024-01-17,ACC-CIRC-C,ACC-CIRC-A,14000,14000.0,EUR,TRANSFER,0,FRANCE,SPAIN,0
```

`data/processed/transactions_clean.csv`

Generated processed CSV from the local pipeline run; useful as a study artifact.

- Row count: 21

- Columns: transaction_id, transaction_timestamp, transaction_date, sender_account_id, receiver_account_id, amount_original, amount_eur, currency, transaction_type, is_laundering

Full CSV content:

```
transaction_id,transaction_timestamp,transaction_date,sender_account_id,receiver_account_id,amount_original,amou
nt_eur,currency,transaction_type,is_laundering
TX-0001,2024-01-02 09:15:00+00:00,2024-01-02,ACC-ORIGIN,ACC-SMURF,52000,52000.0,EUR,WIRE_IN,0
TX-0002,2024-01-02 12:10:00+00:00,2024-01-02,ACC-SMURF,ACC-BEN-01,9800,9800.0,EUR,TRANSFER,0
TX-0003,2024-01-02 15:20:00+00:00,2024-01-02,ACC-SMURF,ACC-BEN-02,9500,9500.0,EUR,TRANSFER,0
TX-0004,2024-01-03 09:40:00+00:00,2024-01-03,ACC-SMURF,ACC-BEN-03,8700,8700.0,EUR,TRANSFER,0
TX-0005,2024-01-03 13:35:00+00:00,2024-01-03,ACC-SMURF,ACC-BEN-04,9100,9100.0,EUR,TRANSFER,0
TX-0006,2024-01-04 08:05:00+00:00,2024-01-04,ACC-SMURF,ACC-BEN-05,8900,8900.0,EUR,TRANSFER,0
TX-0007,2024-01-08 10:00:00+00:00,2024-01-08,ACC-SEND-01,ACC-FANIN,4100,4100.0,EUR,TRANSFER,0
TX-0008,2024-01-08 11:10:00+00:00,2024-01-08,ACC-SEND-02,ACC-FANIN,3900,3900.0,EUR,TRANSFER,0
TX-0009,2024-01-08 12:25:00+00:00,2024-01-08,ACC-SEND-03,ACC-FANIN,4200,4200.0,EUR,TRANSFER,0
TX-0010,2024-01-08 14:00:00+00:00,2024-01-08,ACC-SEND-04,ACC-FANIN,4050,4050.0,EUR,TRANSFER,0
TX-0011,2024-01-08 16:30:00+00:00,2024-01-08,ACC-SEND-05,ACC-FANIN,3950,3950.0,EUR,TRANSFER,0
TX-0012,2024-01-10 09:00:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-01,10300,10300.0,EUR,TRANSFER,0
TX-0013,2024-01-10 10:15:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-02,10550,10550.0,EUR,TRANSFER,0
TX-0014,2024-01-10 11:45:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-03,10700,10700.0,EUR,TRANSFER,0
TX-0015,2024-01-10 13:20:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-04,10400,10400.0,EUR,TRANSFER,0
TX-0016,2024-01-10 15:55:00+00:00,2024-01-10,ACC-FANOUT,ACC-REC-05,10650,10650.0,EUR,TRANSFER,0
TX-0017,2024-01-12 09:30:00+00:00,2024-01-12,ACC-HR-01,ACC-GEO,12500,12500.0,EUR,TRANSFER,0
TX-0018,2024-01-12 15:45:00+00:00,2024-01-12,ACC-GEO,ACC-HR-02,11800,11800.0,EUR,TRANSFER,0
TX-0019,2024-01-15 10:00:00+00:00,2024-01-15,ACC-CIRC-A,ACC-CIRC-B,15000,15000.0,EUR,TRANSFER,0
TX-0020,2024-01-16 10:00:00+00:00,2024-01-16,ACC-CIRC-B,ACC-CIRC-C,14500,14500.0,EUR,TRANSFER,0
TX-0021,2024-01-17 10:00:00+00:00,2024-01-17,ACC-CIRC-C,ACC-CIRC-A,14000,14000.0,EUR,TRANSFER,0
```

`data/processed/transactions_ingested.csv`

Generated processed CSV from the local pipeline run; useful as a study artifact.

- Row count: 21

- Columns: transaction_id, timestamp, sender_account_id, receiver_account_id, amount, currency, transaction_type, is_laundering, ingested_at

Full CSV content:

```
transaction_id,timestamp,sender_account_id,receiver_account_id,amount,currency,transaction_type,is_laundering,in
gested_at
TX-0001,2024-01-02T09:15:00Z,ACC-ORIGIN,ACC-SMURF,52000,EUR,WIRE_IN,0,2026-04-26T14:37:11.041701+00:00
TX-0002,2024-01-02T12:10:00Z,ACC-SMURF,ACC-BEN-01,9800,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0003,2024-01-02T15:20:00Z,ACC-SMURF,ACC-BEN-02,9500,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0004,2024-01-03T09:40:00Z,ACC-SMURF,ACC-BEN-03,8700,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0005,2024-01-03T13:35:00Z,ACC-SMURF,ACC-BEN-04,9100,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0006,2024-01-04T08:05:00Z,ACC-SMURF,ACC-BEN-05,8900,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0007,2024-01-08T10:00:00Z,ACC-SEND-01,ACC-FANIN,4100,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0008,2024-01-08T11:10:00Z,ACC-SEND-02,ACC-FANIN,3900,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0009,2024-01-08T12:25:00Z,ACC-SEND-03,ACC-FANIN,4200,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0010,2024-01-08T14:00:00Z,ACC-SEND-04,ACC-FANIN,4050,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0011,2024-01-08T16:30:00Z,ACC-SEND-05,ACC-FANIN,3950,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0012,2024-01-10T09:00:00Z,ACC-FANOUT,ACC-REC-01,10300,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0013,2024-01-10T10:15:00Z,ACC-FANOUT,ACC-REC-02,10550,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0014,2024-01-10T11:45:00Z,ACC-FANOUT,ACC-REC-03,10700,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0015,2024-01-10T13:20:00Z,ACC-FANOUT,ACC-REC-04,10400,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0016,2024-01-10T15:55:00Z,ACC-FANOUT,ACC-REC-05,10650,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0017,2024-01-12T09:30:00Z,ACC-HR-01,ACC-GEO,12500,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0018,2024-01-12T15:45:00Z,ACC-GEO,ACC-HR-02,11800,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0019,2024-01-15T10:00:00Z,ACC-CIRC-A,ACC-CIRC-B,15000,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0020,2024-01-16T10:00:00Z,ACC-CIRC-B,ACC-CIRC-C,14500,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
TX-0021,2024-01-17T10:00:00Z,ACC-CIRC-C,ACC-CIRC-A,14000,EUR,TRANSFER,0,2026-04-26T14:37:11.041701+00:00
```

`outputs/alerts_sample.csv`

Final sample alerts table generated by the rule-based pipeline.

- Row count: 6

- Columns: alert_id, account_id, rule_name, severity, reason, metric_1_name, metric_1_value, metric_2_name, metric_2_value, detection_window_start, detection_window_end, detection_date

Full CSV content:

```
alert_id,account_id,rule_name,severity,reason,metric_1_name,metric_1_value,metric_2_name,metric_2_value,detectio
n_window_start,detection_window_end,detection_date
fe7744a7b196,ACC-GEO,HIGH_RISK_GEOGRAPHY,LOW,"Account transacted with a training watchlist country (IRAN,
MYANMAR); this requires further context review.",high_risk_transfer_count,2,high_risk_amount_eur,24300.0,2024-
01-12T09:30:00+00:00,2024-01-12T15:45:00+00:00,2024-01-12
75f3fe033f67,ACC-HR-01,HIGH_RISK_GEOGRAPHY,LOW,Account transacted with a training watchlist country (IRAN); this
 requires further context review.,high_risk_transfer_count,1,high_risk_amount_eur,12500.0,2024-01-
12T09:30:00+00:00,2024-01-12T09:30:00+00:00,2024-01-12
439a4618fa32,ACC-HR-02,HIGH_RISK_GEOGRAPHY,LOW,Account transacted with a training watchlist country (MYANMAR);
this requires further context review.,high_risk_transfer_count,1,high_risk_amount_eur,11800.0,2024-01-
12T15:45:00+00:00,2024-01-12T15:45:00+00:00,2024-01-12
315b28414d1a,ACC-FANIN,FAN_IN,MEDIUM,Account received funds from 5 unique counterparties within 24 hours; this
may indicate unusual movement for review.,unique_counterparties_24h,5,amount_24h_eur,20200.0,2024-01-
08T10:00:00+00:00,2024-01-08T16:30:00+00:00,2024-01-08
a814bec7113b,ACC-FANOUT,FAN_OUT,MEDIUM,Account sent funds to 5 unique counterparties within 24 hours; this may
indicate unusual movement for review.,unique_counterparties_24h,5,amount_24h_eur,52600.0,2024-01-
10T09:00:00+00:00,2024-01-10T15:55:00+00:00,2024-01-10
cfcd2d041e2f,ACC-SMURF,SMURFING,MEDIUM,Account sent 5 transfers below EUR 9999 within 72 hours; this may
indicate structuring and requires review.,transaction_count_72h,5,amount_72h_eur,46000.0,2024-01-
02T12:10:00+00:00,2024-01-04T08:05:00+00:00,2024-01-04
```

9. Experimental / Archived Content

The files below are optional/non-core. They preserve previous or advanced work, but the main interview story should stay focused on the simplified CSV/Pandas/SQL rule-based pipeline.

`experimental/README.md`

Optional/non-core file retained for archived advanced work or future improvements.

```
# Experimental Work

Esta carpeta conserva componentes avanzados del proyecto original. Se mantienen para aprendizaje futuro, pero no
 forman parte del flujo principal del portfolio junior.

Contenido:

- `ml_scoring/`: experimentos de scoring con modelos.
- `graph_analytics/`: carga y analisis de red.
- `orchestration/`: Airflow, dbt, Docker y carga avanzada a base de datos.
- `sql_investigations/`: consultas exploratorias mas extensas.
- `archive_docs/`: documentacion visual anterior.
La ruta recomendada para explicar el proyecto en entrevista es la del README principal: CSV, Pandas, SQL,
```

```
features basicas y alertas por reglas.
```

`experimental/graph_analytics/README.md`

Optional/non-core file retained for archived advanced work or future improvements.

```
# Optional Graph Analytics

Esta carpeta conserva el trabajo de analisis de red como mejora futura.

No forma parte del flujo principal del proyecto simplificado. Para la version junior, basta con explicar
contrapartes unicas, paises relacionados y reglas fan-in/fan-out.

Ideas de aprendizaje futuro:

- cargar cuentas y transferencias como grafo
- analizar comunidades y centralidad
- explorar rutas entre cuentas
- comparar patrones de red con alertas por reglas
```

`experimental/graph_analytics/load_graph.py`

Optional/non-core file retained for archived advanced work or future improvements.

```
from __future__ import annotations

import sys
from datetime import UTC, datetime
from pathlib import Path

import pandas as pd
from neo4j import GraphDatabase
from sqlalchemy import text

sys.path.append(str(Path(__file__).resolve().parents[1] / "orchestration" / "legacy_postgres_ingestion"))

from common import configure_logging, env, get_engine


LOGGER = configure_logging("amlguardian.graph")
BATCH_SIZE = 10_000


def neo4j_driver():
    return GraphDatabase.driver(
        env("NEO4J_URI", "bolt://neo4j:7687"),
        auth=(env("NEO4J_USERNAME", required=True), env("NEO4J_PASSWORD", required=True)),
    )


def reset_graph(session) -> None:
    session.run(
        "CREATE CONSTRAINT account_id_unique IF NOT EXISTS FOR (a:Account) REQUIRE a.id IS UNIQUE"
    )
    session.run("MATCH (n) DETACH DELETE n")


def load_accounts(session, accounts: list[dict]) -> None:
    session.run(
        """
        UNWIND $rows AS row
        MERGE (a:Account {id: row.account_id})
        SET a.name = row.account_name,
            a.country = row.country,
            a.account_type = row.account_type,
            a.risk_score = coalesce(a.risk_score, 0.0)
        """,
        rows=accounts,
    )


def load_transactions(session, transactions: list[dict]) -> None:
    session.run(
        """
        UNWIND $rows AS row
        MATCH (s:Account {id: row.sender_account_id})
        MATCH (r:Account {id: row.receiver_account_id})
        CREATE (s)-[:TRANSFERRED {
            transaction_id: row.transaction_id,
            amount_eur: row.amount_eur,
            currency: row.currency,
            timestamp: datetime(row.transaction_timestamp_utc),
            is_laundering: row.is_laundering
        }]->(r)
        """,
        rows=transactions,
    )


def build_graph_projection(session) -> None:
    exists = session.run(
        "CALL gds.graph.exists('amlguardian_graph') YIELD exists RETURN exists"
    ).single()
```

```
    if exists and exists["exists"]:
        session.run("CALL gds.graph.drop('amlguardian_graph', false)")

    session.run(
        """
        CALL gds.graph.project(
            'amlguardian_graph',
            'Account',
            {
                TRANSFERRED: {
                    orientation: 'NATURAL',
                    properties: ['amount_eur', 'is_laundering']
                }
            }
        )
        """
    )
    session.run(
        """
        CALL gds.pageRank.write(
            'amlguardian_graph',
            {
                writeProperty: 'pagerank_score',
                maxIterations: 50,
                dampingFactor: 0.85
            }
        )
        """
    )
    session.run(
        """
        CALL gds.louvain.write(
            'amlguardian_graph',
            {
                writeProperty: 'community_id'
            }
        )
        """
    )


def fetch_graph_metrics(session) -> pd.DataFrame:
    result = session.run(
        """
        MATCH (a:Account)
        RETURN a.id AS account_id,
               coalesce(a.pagerank_score, 0.0) AS pagerank_score,
               coalesce(a.community_id, -1) AS community_id
        ORDER BY pagerank_score DESC
        """
    )
    rows = [record.data() for record in result]
    metrics = pd.DataFrame(rows)
    metrics["refreshed_at"] = datetime.now(tz=UTC)
    return metrics


def persist_graph_metrics(metrics: pd.DataFrame) -> None:
    engine = get_engine()
    with engine.begin() as connection:
        connection.execute(text("CREATE SCHEMA IF NOT EXISTS ml"))
        connection.execute(text("DROP TABLE IF EXISTS ml.graph_account_metrics"))
    metrics.to_sql(
        "graph_account_metrics",
        engine,
        schema="ml",
        if_exists="replace",
        index=False,
        method="multi",
        chunksize=5_000,
    )


def main() -> None:
    engine = get_engine()
    driver = neo4j_driver()
    LOGGER.info("Loading staged account and transaction network into Neo4j")

    with driver.session(database=env("NEO4J_DATABASE", "neo4j")) as session:
        reset_graph(session)

        for accounts_chunk in pd.read_sql(
            """
            SELECT account_id, account_name, country, account_type
            FROM staging.stg_accounts
            ORDER BY account_id
            """,
            engine,
            chunksize=BATCH_SIZE,
        ):
            load_accounts(session, accounts_chunk.to_dict("records"))
        LOGGER.info("Account nodes loaded into Neo4j")

        for transactions_chunk in pd.read_sql(
            """
```

```
            SELECT transaction_id,
                   sender_account_id,
                   receiver_account_id,
                   amount_eur,
                   currency,
                   to_char(transaction_timestamp_utc AT TIME ZONE 'UTC', 'YYYY-MM-DD\"T\"HH24:MI:SS') || 'Z' AS
transaction_timestamp_utc,
                   is_laundering
            FROM staging.stg_transactions
            ORDER BY transaction_timestamp_utc
            """,
            engine,
            chunksize=BATCH_SIZE,
        ):
            load_transactions(session, transactions_chunk.to_dict("records"))
        LOGGER.info("Transaction relationships loaded into Neo4j")

        build_graph_projection(session)
        metrics = fetch_graph_metrics(session)

    persist_graph_metrics(metrics)
    top_accounts = metrics.sort_values("pagerank_score", ascending=False).head(20)
    LOGGER.info("Top 20 accounts by PageRank:\n%s", top_accounts.to_string(index=False))
    driver.close()
    LOGGER.info("Neo4j graph load completed successfully")


if __name__ == "__main__":
    main()
```

`experimental/ml_scoring/README.md`

Optional/non-core file retained for archived advanced work or future improvements.

```
# Optional ML Scoring

Esta carpeta conserva el trabajo de machine learning como posible mejora futura.

No forma parte del proyecto principal. Para un portfolio junior, la deteccion principal se explica mediante
reglas simples y trazables.

Ideas que podrian explorarse mas adelante:

- entrenar un modelo de riesgo con features de cuenta
- comparar modelos supervisados y no supervisados
- anadir explicabilidad de modelos
- usar feedback de analistas como etiqueta de entrenamiento

Antes de retomar esta parte, conviene revisar dependencias, datos de entrenamiento y validacion.
```

`experimental/ml_scoring/train_model.py`

Optional/non-core file retained for archived advanced work or future improvements.

```
from __future__ import annotations

import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import shap
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline
from lightgbm import LGBMClassifier
from sklearn.ensemble import IsolationForest
from sklearn.metrics import average_precision_score, confusion_matrix, f1_score
from sklearn.model_selection import train_test_split
from sqlalchemy import text
from xgboost import XGBClassifier

sys.path.append(str(Path(__file__).resolve().parents[1] / "orchestration" / "legacy_postgres_ingestion"))

from common import configure_logging, env, ensure_directory, get_engine


LOGGER = configure_logging("amlguardian.ml")
RANDOM_STATE = 42


def load_feature_dataset() -> pd.DataFrame:
    engine = get_engine()
    dataset = pd.read_sql(
        """
        SELECT *
        FROM features.fct_account_features
        ORDER BY account_id
        """,
        engine,
    )
    LOGGER.info("Loaded %s feature rows from Postgres", len(dataset))
    return dataset
```

```
def build_supervised_models() -> dict[str, Pipeline]:
    return {
        "xgboost": Pipeline(
            steps=[
                ("smote", SMOTE(random_state=RANDOM_STATE)),
                (
                    "model",
                    XGBClassifier(
                        objective="binary:logistic",
                        eval_metric="logloss",
                        n_estimators=300,
                        learning_rate=0.05,
                        max_depth=5,
                        subsample=0.85,
                        colsample_bytree=0.85,
                        min_child_weight=2,
                        reg_lambda=1.0,
                        random_state=RANDOM_STATE,
                        n_jobs=-1,
                    ),
                ),
            ]
        ),
        "lightgbm": Pipeline(
            steps=[
                ("smote", SMOTE(random_state=RANDOM_STATE)),
                (
                    "model",
                    LGBMClassifier(
                        n_estimators=300,
                        learning_rate=0.05,
                        num_leaves=31,
                        subsample=0.85,
                        colsample_bytree=0.85,
                        random_state=RANDOM_STATE,
                        n_jobs=-1,
                    ),
                ),
            ]
        ),
    }


def evaluate_supervised_model(model: Pipeline, x_train, x_test, y_train, y_test) -> dict:
    model.fit(x_train, y_train)
    probabilities = model.predict_proba(x_test)[:, 1]
    predictions = (probabilities >= 0.5).astype(int)
    return {
        "pr_auc": float(average_precision_score(y_test, probabilities)),
        "f1": float(f1_score(y_test, predictions, zero_division=0)),
        "confusion_matrix": confusion_matrix(y_test, predictions).tolist(),
        "probabilities": probabilities,
        "predictions": predictions,
        "estimator": model,
    }


def evaluate_isolation_forest(x_train, x_test, y_test, contamination: float) -> dict:
    model = IsolationForest(
        n_estimators=300,
        contamination=max(contamination, 0.001),
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )
    model.fit(x_train)
    anomaly_score = -model.decision_function(x_test)
    scaled_score = (anomaly_score - anomaly_score.min()) / (
        (anomaly_score.max() - anomaly_score.min()) or 1.0
    )
    predictions = (model.predict(x_test) == -1).astype(int)
    return {
        "pr_auc": float(average_precision_score(y_test, scaled_score)),
        "f1": float(f1_score(y_test, predictions, zero_division=0)),
        "confusion_matrix": confusion_matrix(y_test, predictions).tolist(),
        "probabilities": scaled_score,
        "predictions": predictions,
        "estimator": model,
    }


def generate_shap_outputs(model: Pipeline, features: pd.DataFrame, output_dir: Path) -> list[str]:
    classifier = model.named_steps["model"]
    explainer = shap.TreeExplainer(classifier)
    shap_values = explainer.shap_values(features)

    summary_sample = features.sample(min(len(features), 10_000), random_state=RANDOM_STATE)
    shap_sample = explainer.shap_values(summary_sample)
    plt.figure(figsize=(12, 7))
    shap.summary_plot(shap_sample, summary_sample, show=False)
    plt.tight_layout()
    shap_path = output_dir / "shap_summary.png"
    plt.savefig(shap_path, dpi=180, bbox_inches="tight")
    plt.close()
    LOGGER.info("Saved SHAP summary plot to %s", shap_path)
    top_features = []
```

```
    for row_values in shap_values:
        contributions = (
            pd.Series(row_values, index=features.columns)
            .abs()
            .sort_values(ascending=False)
            .head(3)
            .index.tolist()
        )
        top_features.append("|".join(contributions))
    return top_features


def assign_risk_tier(score: float) -> str:
    if score >= 75:
        return "CRITICAL"
    if score >= 50:
        return "HIGH"
    if score >= 25:
        return "MEDIUM"
    return "LOW"


def persist_results(predictions: pd.DataFrame, metrics: dict, output_dir: Path) -> None:
    engine = get_engine()
    output_dir.mkdir(parents=True, exist_ok=True)

    csv_path = output_dir / "risk_scores.csv"
    predictions[["account_id", "risk_score", "risk_tier", "top_3_features"]].to_csv(
        csv_path,
        index=False,
    )
    LOGGER.info("Saved risk scores to %s", csv_path)

    metrics_path = output_dir / "model_metrics.json"
    metrics_path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
    LOGGER.info("Saved model benchmark metrics to %s", metrics_path)

    with engine.begin() as connection:
        connection.execute(text("CREATE SCHEMA IF NOT EXISTS ml"))
        connection.execute(text("DROP TABLE IF EXISTS ml.risk_scores"))
    predictions.to_sql(
        "risk_scores",
        engine,
        schema="ml",
        if_exists="replace",
        index=False,
        method="multi",
        chunksize=5_000,
    )


def main() -> None:
    output_dir = ensure_directory(env("OUTPUT_DIR", "outputs"))
    dataset = load_feature_dataset().fillna(0)

    target = dataset["is_laundering_any"].astype(int)
    features = dataset.drop(columns=["account_id", "is_laundering_any"])

    x_train, x_test, y_train, y_test = train_test_split(
        features,
        target,
        test_size=0.25,
        random_state=RANDOM_STATE,
        stratify=target,
    )

    contamination = float(y_train.mean())
    benchmark_results: dict[str, dict] = {}
    supervised_models = build_supervised_models()

    for model_name, model in supervised_models.items():
        LOGGER.info("Training %s benchmark", model_name)
        evaluation = evaluate_supervised_model(model, x_train, x_test, y_train, y_test)
        benchmark_results[model_name] = {
            "pr_auc": evaluation["pr_auc"],
            "f1": evaluation["f1"],
            "confusion_matrix": evaluation["confusion_matrix"],
        }

    LOGGER.info("Training isolation_forest benchmark")
    isolation_evaluation = evaluate_isolation_forest(x_train, x_test, y_test, contamination)
    benchmark_results["isolation_forest"] = {
        "pr_auc": isolation_evaluation["pr_auc"],
        "f1": isolation_evaluation["f1"],
        "confusion_matrix": isolation_evaluation["confusion_matrix"],
    }

    primary_model = supervised_models["xgboost"]
    primary_model.fit(features, target)
    probabilities = primary_model.predict_proba(features)[:, 1]
    top_features = generate_shap_outputs(primary_model, features, output_dir)

    predictions = pd.DataFrame(
        {
            "account_id": dataset["account_id"],
```

```
            "risk_score": (probabilities * 100).round(2),
            "risk_tier": [assign_risk_tier(score) for score in probabilities * 100],
            "top_3_features": top_features,
        }
    )
    predictions["model_name"] = "xgboost"
    predictions["scored_at"] = pd.Timestamp.utcnow()

    persist_results(predictions, benchmark_results, output_dir)
    LOGGER.info("Model training and scoring completed successfully")
    LOGGER.info("Benchmark summary: %s", json.dumps(benchmark_results, indent=2))


if __name__ == "__main__":
    main()
```

`experimental/orchestration/.env.example`

Optional/non-core file retained for archived advanced work or future improvements.

```
COMPOSE_PROJECT_NAME=amlguardian

POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=amlguardian
POSTGRES_USER=aml
POSTGRES_PASSWORD=CHANGE_ME_POSTGRES_PASSWORD

PGADMIN_DEFAULT_EMAIL=admin@example.com
PGADMIN_DEFAULT_PASSWORD=CHANGE_ME_PGADMIN_PASSWORD
PGADMIN_PORT=5050

AIRFLOW_UID=50000
AIRFLOW_ADMIN_USERNAME=admin
AIRFLOW_ADMIN_PASSWORD=CHANGE_ME_AIRFLOW_ADMIN_PASSWORD
AIRFLOW_ADMIN_EMAIL=admin@example.com
AIRFLOW_FERNET_KEY=GENERATE_A_VALID_FERNET_KEY
AIRFLOW_WEBSERVER_SECRET_KEY=CHANGE_ME_AIRFLOW_WEBSERVER_SECRET
AIRFLOW_PORT=8080

NEO4J_HTTP_PORT=7474
NEO4J_BOLT_PORT=7687
NEO4J_URI=bolt://neo4j:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=CHANGE_ME_NEO4J_PASSWORD
NEO4J_DATABASE=neo4j

AMLGUARDIAN_ROOT=/opt/amlguardian
DATA_DIR=/opt/amlguardian/data
TRANSACTIONS_CSV=/opt/amlguardian/data/transactions.csv
ACCOUNTS_CSV=/opt/amlguardian/data/accounts.csv
OUTPUT_DIR=/opt/amlguardian/outputs
DBT_PROFILES_DIR=/opt/amlguardian/dbt
```

`experimental/orchestration/README.md`

Optional/non-core file retained for archived advanced work or future improvements.

```
# Optional Orchestration and Database Stack

Esta carpeta conserva la version avanzada con Airflow, dbt, Docker y PostgreSQL.

No es necesaria para ejecutar el proyecto principal. El flujo recomendado esta en `src/` y se ejecuta con
comandos Python sencillos.

Esta parte puede servir como trabajo futuro si se quiere practicar:

- ejecucion programada
- transformaciones dbt
- carga en PostgreSQL
- contenedores Docker
- separacion de capas raw, staging y marts
```

`experimental/orchestration/dags/amlguardian_pipeline.py`

Optional/non-core file retained for archived advanced work or future improvements.

```
from __future__ import annotations

import logging
import os
from datetime import datetime, timedelta

import pendulum
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from sqlalchemy import create_engine, text


LOGGER = logging.getLogger("amlguardian.airflow")
PROJECT_DIR = "/opt/amlguardian"
```

```
DBT_DIR = f"{PROJECT_DIR}/dbt"


def postgres_url() -> str:
    return (
        "postgresql+psycopg2://"
        f"{os.environ['POSTGRES_USER']}:{os.environ['POSTGRES_PASSWORD']}"
        f"@postgres:5432/{os.environ['POSTGRES_DB']}"
    )


def notify_completion() -> None:
    engine = create_engine(postgres_url(), future=True)
    queries = {
        "raw_transactions": "select count(*) from raw.transactions",
        "feature_accounts": "select count(*) from features.fct_account_features",
        "alerts": "select count(*) from alerts.fct_alerts",
        "sar_candidates": "select count(*) from alerts.fct_sar_candidates",
        "risk_scores": "select count(*) from ml.risk_scores",
        "graph_metrics": "select count(*) from ml.graph_account_metrics",
    }

    summary = {}
    with engine.connect() as connection:
        for label, query in queries.items():
            summary[label] = connection.execute(text(query)).scalar_one()

    LOGGER.info("AMLGuardian pipeline completed successfully")
    LOGGER.info("Pipeline summary: %s", summary)


default_args = {
    "owner": "amlguardian",
    "depends_on_past": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}


with DAG(
    dag_id="amlguardian_pipeline",
    description="End-to-end AMLGuardian monitoring pipeline: ingestion, dbt, graph analytics, and ML scoring.",
    default_args=default_args,
    start_date=pendulum.datetime(2026, 1, 1, tz="Europe/Madrid"),
    schedule="0 2 * * *",
    catchup=False,
    max_active_runs=1,
    tags=["aml", "fincrime", "portfolio"],
) as dag:
    common_env = {
        **os.environ,
        "DBT_PROFILES_DIR": DBT_DIR,
        "PYTHONPATH": f"/opt/airflow:{PROJECT_DIR}",
    }

    ingest_raw_data = BashOperator(
        task_id="ingest_raw_data",
        env=common_env,
        bash_command=f"python {PROJECT_DIR}/scripts/ingest.py",
    )

    run_dbt_staging = BashOperator(
        task_id="run_dbt_staging",
        env=common_env,
        bash_command=f"dbt run --project-dir {DBT_DIR} --profiles-dir {DBT_DIR} --select staging",
    )

    run_dbt_features = BashOperator(
        task_id="run_dbt_features",
        env=common_env,
        bash_command=f"dbt run --project-dir {DBT_DIR} --profiles-dir {DBT_DIR} --select marts.features",
    )

    run_dbt_alerts = BashOperator(
        task_id="run_dbt_alerts",
        env=common_env,
        bash_command=f"dbt run --project-dir {DBT_DIR} --profiles-dir {DBT_DIR} --select marts.alerts",
    )

    run_dbt_tests = BashOperator(
        task_id="run_dbt_tests",
        env=common_env,
        bash_command=(
            f"dbt test --project-dir {DBT_DIR} --profiles-dir {DBT_DIR} "
            f"&& dbt docs generate --project-dir {DBT_DIR} --profiles-dir {DBT_DIR}"
        ),
    )

    load_neo4j_graph = BashOperator(
        task_id="load_neo4j_graph",
        env=common_env,
        bash_command=f"python {PROJECT_DIR}/scripts/load_graph.py",
    )

    train_ml_model = BashOperator(
```

```
        task_id="train_ml_model",
        env=common_env,
        bash_command=f"python {PROJECT_DIR}/scripts/train_model.py",
    )

    notify_completion_task = PythonOperator(
        task_id="notify_completion",
        python_callable=notify_completion,
    )

    (
        ingest_raw_data
        >> run_dbt_staging
        >> run_dbt_features
        >> run_dbt_alerts
        >> run_dbt_tests
        >> load_neo4j_graph
        >> train_ml_model
        >> notify_completion_task
    )
```

`experimental/orchestration/dbt/dbt_project.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
name: amlguardian
version: 1.0.0
config-version: 2

profile: amlguardian

model-paths: ["models"]
test-paths: ["tests"]
macro-paths: ["macros"]
target-path: "target"
clean-targets:
  - "target"
  - "dbt_packages"
  - "logs"

vars:
  alert_lookback_days: 90

models:
  amlguardian:
    +persist_docs:
      relation: true
      columns: true
    staging:
      +schema: staging
      +materialized: table
    marts:
      features:
        +schema: features
        +materialized: table
      alerts:
        +schema: alerts
        +materialized: table
```

`experimental/orchestration/dbt/models/marts/alerts/alert_circular.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
-- Lookback window controlled by dbt variable alert_lookback_days (default: 90).
-- Override at runtime: dbt run --vars '{"alert_lookback_days": 180}'
with recursive paths as (
    select
        t.sender_account_id as root_account_id,
        t.receiver_account_id as current_account_id,
        array[t.sender_account_id, t.receiver_account_id]::text[] as account_path,
        array[t.transaction_id]::text[] as transaction_path,
        t.transaction_timestamp_utc as start_ts,
        t.transaction_timestamp_utc as last_ts,
        t.amount_eur as total_amount,
        1 as depth
    from {{ ref('stg_transactions') }} t
    where t.sender_account_id <> t.receiver_account_id
      and t.transaction_timestamp_utc >= current_timestamp - interval '{{ var("alert_lookback_days") }} days'

    union all

    select
        p.root_account_id,
        t.receiver_account_id as current_account_id,
        p.account_path || t.receiver_account_id,
        p.transaction_path || t.transaction_id,
        p.start_ts,
        t.transaction_timestamp_utc as last_ts,
        p.total_amount + t.amount_eur as total_amount,
        p.depth + 1 as depth
    from paths p
    join {{ ref('stg_transactions') }} t
        on t.sender_account_id = p.current_account_id
```

```
       and t.transaction_timestamp_utc >= p.last_ts
       and t.transaction_timestamp_utc <= p.start_ts + interval '7 days'
    where p.depth < 3
      and (
            (p.depth = 2 and t.receiver_account_id = p.root_account_id)
         or (p.depth < 2 and not t.receiver_account_id = any(p.account_path))
      )
),
qualified_cycles as (
    select
        root_account_id as account_id,
        account_path,
        transaction_path,
        start_ts,
        last_ts as detected_at,
        total_amount,
        row_number() over (
            partition by root_account_id
            order by total_amount desc, last_ts asc
        ) as rn
    from paths
    where depth = 3
      and current_account_id = root_account_id
      and cardinality(account_path) = 4
)

select
    md5(account_id || '|CIRCULAR|' || array_to_string(transaction_path, '|')) as alert_id,
    account_id,
    'CIRCULAR' as alert_type,
    case
        when total_amount >= 100000 then 'CRITICAL'
        when total_amount >= 50000 then 'HIGH'
        else 'MEDIUM'
    end as severity,
    detected_at,
    round(total_amount, 2) as amount_involved,
    'Circular flow detected across accounts ' || account_path[1] || ' -> ' ||
    account_path[2] || ' -> ' || account_path[3] || ' -> ' || account_path[4] ||
    ' within 7 days, recycling EUR ' || round(total_amount, 2) || '.' as description
from qualified_cycles
where rn = 1
```

`experimental/orchestration/dbt/models/marts/alerts/alert_fan_in.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with anchored_windows as (
    select
        a.receiver_account_id as account_id,
        a.transaction_id as anchor_transaction_id,
        a.transaction_timestamp_utc as window_start,
        max(b.transaction_timestamp_utc) as window_end,
        count(*) as txn_count_24h,
        count(distinct b.sender_account_id) as distinct_senders_24h,
        sum(b.amount_eur) as amount_24h
    from {{ ref('stg_transactions') }} a
    join {{ ref('stg_transactions') }} b
        on a.receiver_account_id = b.receiver_account_id
       and b.transaction_timestamp_utc between a.transaction_timestamp_utc
           and a.transaction_timestamp_utc + interval '24 hours'
    group by 1, 2, 3
    having count(distinct b.sender_account_id) >= 10
),
ranked as (
    select
        *,
        row_number() over (
            partition by account_id
            order by distinct_senders_24h desc, amount_24h desc, window_start asc
        ) as rn
    from anchored_windows
)

select
    md5(account_id || '|FAN_IN|' || anchor_transaction_id) as alert_id,
    account_id,
    'FAN_IN' as alert_type,
    case
        when distinct_senders_24h >= 20 then 'CRITICAL'
        when distinct_senders_24h >= 15 then 'HIGH'
        else 'MEDIUM'
    end as severity,
    window_end as detected_at,
    round(amount_24h, 2) as amount_involved,
    'Account ' || account_id || ' received funds from ' || distinct_senders_24h ||
    ' unique originators within 24 hours between ' || window_start || ' and ' ||
    window_end || ', suggesting consolidation activity or a mule collection node.' as description
from ranked
where rn = 1
```

`experimental/orchestration/dbt/models/marts/alerts/alert_fan_out.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with anchored_windows as (
    select
        a.sender_account_id as account_id,
        a.transaction_id as anchor_transaction_id,
        a.transaction_timestamp_utc as window_start,
        max(b.transaction_timestamp_utc) as window_end,
        count(*) as txn_count_24h,
        count(distinct b.receiver_account_id) as distinct_receivers_24h,
        sum(b.amount_eur) as amount_24h
    from {{ ref('stg_transactions') }} a
    join {{ ref('stg_transactions') }} b
        on a.sender_account_id = b.sender_account_id
       and b.transaction_timestamp_utc between a.transaction_timestamp_utc
           and a.transaction_timestamp_utc + interval '24 hours'
    group by 1, 2, 3
    having count(distinct b.receiver_account_id) >= 10
),
ranked as (
    select
        *,
        row_number() over (
            partition by account_id
            order by distinct_receivers_24h desc, amount_24h desc, window_start asc
        ) as rn
    from anchored_windows
)

select
    md5(account_id || '|FAN_OUT|' || anchor_transaction_id) as alert_id,
    account_id,
    'FAN_OUT' as alert_type,
    case
        when distinct_receivers_24h >= 20 then 'CRITICAL'
        when distinct_receivers_24h >= 15 then 'HIGH'
        else 'MEDIUM'
    end as severity,
    window_end as detected_at,
    round(amount_24h, 2) as amount_involved,
    'Account ' || account_id || ' distributed funds to ' || distinct_receivers_24h ||
    ' unique beneficiaries within 24 hours between ' || window_start || ' and ' ||
    window_end || ', a classic fan-out dispersion pattern.' as description
from ranked
where rn = 1
```

`experimental/orchestration/dbt/models/marts/alerts/alert_high_risk_geography.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with high_risk_countries as (
    select country
    from (
        values
            ('IRAN'),
            ('DPRK'),
            ('MYANMAR'),
            ('SYRIA'),
            ('YEMEN'),
            ('HAITI'),
            ('SOUTH SUDAN'),
            ('NIGERIA'),
            ('VENEZUELA'),
            ('CAMEROON'),
            ('CROATIA'),
            ('KENYA'),
            ('NAMIBIA'),
            ('VIETNAM'),
            ('SOUTH AFRICA')
    ) as watchlist(country)
),
flagged_transactions as (
    select
        t.transaction_id,
        t.transaction_timestamp_utc,
        t.amount_eur,
        t.sender_account_id,
        t.receiver_account_id,
        s.country as sender_country,
        r.country as receiver_country
    from {{ ref('stg_transactions') }} t
    left join {{ ref('stg_accounts') }} s
        on t.sender_account_id = s.account_id
    left join {{ ref('stg_accounts') }} r
        on t.receiver_account_id = r.account_id
    where s.country in (select country from high_risk_countries)
       or r.country in (select country from high_risk_countries)
),
account_exposure as (
    select
        sender_account_id as account_id,
        transaction_timestamp_utc,
        amount_eur,
        case
```

```
            when sender_country in (select country from high_risk_countries) then sender_country
            else receiver_country
        end as risky_country
    from flagged_transactions

    union all

    select
        receiver_account_id as account_id,
        transaction_timestamp_utc,
        amount_eur,
        case
            when receiver_country in (select country from high_risk_countries) then receiver_country
            else sender_country
        end as risky_country
    from flagged_transactions
),
ranked as (
    select
        account_id,
        max(transaction_timestamp_utc) as detected_at,
        sum(amount_eur) as amount_involved,
        count(*) as exposure_count,
        string_agg(distinct risky_country, ', ' order by risky_country) as risky_countries
    from account_exposure
    group by account_id
)

select
    md5(account_id || '|HIGH_RISK_GEOGRAPHY') as alert_id,
    account_id,
    'HIGH_RISK_GEOGRAPHY' as alert_type,
    case
        when exposure_count >= 10 then 'CRITICAL'
        when exposure_count >= 5 then 'HIGH'
        else 'MEDIUM'
    end as severity,
    detected_at,
    round(amount_involved, 2) as amount_involved,
    'Account ' || account_id || ' transacted with high-risk jurisdictions (' ||
    risky_countries || ') across ' || exposure_count ||
    ' flagged transfers, requiring enhanced due diligence.' as description
from ranked
```

`experimental/orchestration/dbt/models/marts/alerts/alert_layering.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
-- Lookback window controlled by dbt variable alert_lookback_days (default: 90).
-- Override at runtime: dbt run --vars '{"alert_lookback_days": 180}'
with recursive chains as (
    select
        t.transaction_id as root_transaction_id,
        t.sender_account_id as root_account_id,
        t.receiver_account_id as current_account_id,
        array[t.sender_account_id, t.receiver_account_id]::text[] as account_path,
        array[t.transaction_id]::text[] as transaction_path,
        t.transaction_timestamp_utc as start_ts,
        t.transaction_timestamp_utc as last_ts,
        t.amount_eur as previous_amount,
        t.amount_eur as total_amount,
        1 as depth
    from {{ ref('stg_transactions') }} t
    where t.sender_account_id <> t.receiver_account_id
      and t.transaction_timestamp_utc >= current_timestamp - interval '{{ var("alert_lookback_days") }} days'

    union all

    select
        c.root_transaction_id,
        c.root_account_id,
        t.receiver_account_id as current_account_id,
        c.account_path || t.receiver_account_id,
        c.transaction_path || t.transaction_id,
        c.start_ts,
        t.transaction_timestamp_utc as last_ts,
        t.amount_eur as previous_amount,
        c.total_amount + t.amount_eur as total_amount,
        c.depth + 1 as depth
    from chains c
    join {{ ref('stg_transactions') }} t
        on t.sender_account_id = c.current_account_id
       and t.transaction_timestamp_utc >= c.last_ts
       and t.transaction_timestamp_utc <= c.last_ts + interval '48 hours'
       and t.amount_eur between c.previous_amount * 0.85 and c.previous_amount * 0.99
    where c.depth < 5
      and not t.receiver_account_id = any(c.account_path)
),
qualified_chains as (
    select
        root_account_id as account_id,
        account_path,
        transaction_path,
        start_ts,
```

```
        last_ts as detected_at,
        total_amount,
        depth,
        row_number() over (
            partition by root_account_id
            order by depth desc, total_amount desc, last_ts asc
        ) as rn
    from chains
    where depth >= 3
)

select
    md5(account_id || '|LAYERING|' || array_to_string(transaction_path, '|')) as alert_id,
    account_id,
    'LAYERING' as alert_type,
    case
        when depth >= 5 then 'CRITICAL'
        when depth = 4 then 'HIGH'
        else 'MEDIUM'
    end as severity,
    detected_at,
    round(total_amount, 2) as amount_involved,
    'Layering chain detected from ' || account_path[1] || ' through ' ||
    array_to_string(account_path[2:cardinality(account_path)], ' -> ') ||
    ', where each hop retained 85%-99% of the previous amount over ' || depth ||
    ' sequential transfers.' as description
from qualified_chains
where rn = 1
```

`experimental/orchestration/dbt/models/marts/alerts/alert_smurfing.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with candidate_transactions as (
    select
        transaction_id,
        sender_account_id as account_id,
        amount_eur,
        transaction_timestamp_utc
    from {{ ref('stg_transactions') }}
    where amount_eur < 9999
),
rolling_windows as (
    select
        account_id,
        transaction_id,
        transaction_timestamp_utc,
        count(*) over (
            partition by account_id
            order by transaction_timestamp_utc
            range between interval '72 hours' preceding and current row
        ) as txn_count_72h,
        sum(amount_eur) over (
            partition by account_id
            order by transaction_timestamp_utc
            range between interval '72 hours' preceding and current row
        ) as amount_72h,
        min(transaction_timestamp_utc) over (
            partition by account_id
            order by transaction_timestamp_utc
            range between interval '72 hours' preceding and current row
        ) as window_start_72h
    from candidate_transactions
),
ranked as (
    select
        *,
        row_number() over (
            partition by account_id
            order by txn_count_72h desc, amount_72h desc, transaction_timestamp_utc asc
        ) as rn
    from rolling_windows
    where txn_count_72h >= 5
)

select
    md5(account_id || '|SMURFING|' || transaction_id) as alert_id,
    account_id,
    'SMURFING' as alert_type,
    case
        when txn_count_72h >= 15 then 'CRITICAL'
        when txn_count_72h >= 10 then 'HIGH'
        else 'MEDIUM'
    end as severity,
    transaction_timestamp_utc as detected_at,
    round(amount_72h, 2) as amount_involved,
    'Account ' || account_id || ' sent ' || txn_count_72h ||
    ' transactions below EUR 9,999 between ' || window_start_72h ||
    ' and ' || transaction_timestamp_utc ||
    ', indicating structuring activity totalling EUR ' || round(amount_72h, 2) || '.' as description
from ranked
where rn = 1
```

`experimental/orchestration/dbt/models/marts/alerts/fct_alerts.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select * from {{ ref('alert_smurfing') }}
union all
select * from {{ ref('alert_fan_out') }}
union all
select * from {{ ref('alert_fan_in') }}
union all
select * from {{ ref('alert_circular') }}
union all
select * from {{ ref('alert_layering') }}
union all
select * from {{ ref('alert_high_risk_geography') }}
```

`experimental/orchestration/dbt/models/marts/alerts/fct_sar_candidates.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with alert_summary as (
    select
        account_id,
        count(*) as alert_count,
        count(*) filter (where severity = 'CRITICAL') as critical_alert_count,
        min(detected_at) as first_alert_at,
        max(detected_at) as latest_alert_at,
        round(sum(amount_involved), 2) as total_amount_involved,
        case
            when max(case severity
                when 'CRITICAL' then 4
                when 'HIGH' then 3
                when 'MEDIUM' then 2
                else 1
            end) = 4 then 'CRITICAL'
            when max(case severity
                when 'CRITICAL' then 4
                when 'HIGH' then 3
                when 'MEDIUM' then 2
                else 1
            end) = 3 then 'HIGH'
            when max(case severity
                when 'CRITICAL' then 4
                when 'HIGH' then 3
                when 'MEDIUM' then 2
                else 1
            end) = 2 then 'MEDIUM'
            else 'LOW'
        end as max_severity,
        string_agg(distinct alert_type, ', ' order by alert_type) as alert_types
    from {{ ref('fct_alerts') }}
    group by account_id
)

select
    account_id,
    alert_count,
    critical_alert_count,
    max_severity,
    alert_types,
    first_alert_at,
    latest_alert_at,
    total_amount_involved,
    case
        when critical_alert_count > 0 then 'CRITICAL severity alert present'
        else 'Three or more alert typologies accumulated'
    end as sar_rationale
from alert_summary
where alert_count >= 3
   or critical_alert_count > 0
```

`experimental/orchestration/dbt/models/marts/alerts/schema.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
version: 2

models:
  - name: alert_smurfing
    description: Detects structuring behavior where an account repeatedly sends sub-threshold payments inside a
72-hour horizon.
    columns:
      - name: alert_id
        description: Deterministic alert identifier.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account that triggered the smurfing pattern.
        tests:
          - not_null
          - relationships:
```

```
            to: ref('stg_accounts')
            field: account_id
      - name: alert_type
        description: AML typology code for the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['SMURFING']
      - name: severity
        description: Risk-based severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
        description: Timestamp when the suspicious pattern was observed.
        tests:
          - not_null
      - name: amount_involved
        description: EUR amount linked to the suspicious structuring window.
        tests:
          - not_null
      - name: description
        description: Human-readable explanation of why the alert fired.
        tests:
          - not_null

  - name: alert_fan_out
    description: Detects single-origin accounts dispersing money to many beneficiaries in a short time window.
    columns:
      - name: alert_id
        description: Deterministic alert identifier.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account that triggered the fan-out pattern.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_type
        description: AML typology code for the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['FAN_OUT']
      - name: severity
        description: Risk-based severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
        description: Timestamp when the suspicious pattern was observed.
        tests:
          - not_null
      - name: amount_involved
        description: EUR amount linked to the suspicious fan-out window.
        tests:
          - not_null
      - name: description
        description: Human-readable explanation of why the alert fired.
        tests:
          - not_null

  - name: alert_fan_in
    description: Detects sink accounts rapidly aggregating funds from many unrelated originators.
    columns:
      - name: alert_id
        description: Deterministic alert identifier.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account that triggered the fan-in pattern.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_type
        description: AML typology code for the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['FAN_IN']
      - name: severity
        description: Risk-based severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
```

```
        description: Timestamp when the suspicious pattern was observed.
        tests:
          - not_null
      - name: amount_involved
        description: EUR amount linked to the suspicious fan-in window.
        tests:
          - not_null
      - name: description
        description: Human-readable explanation of why the alert fired.
        tests:
          - not_null

  - name: alert_circular
    description: Detects three-hop circular movement where funds return to the originating account inside seven
days.
    columns:
      - name: alert_id
        description: Deterministic alert identifier.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account that triggered the circular flow pattern.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_type
        description: AML typology code for the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['CIRCULAR']
      - name: severity
        description: Risk-based severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
        description: Timestamp when the circular pattern completed.
        tests:
          - not_null
      - name: amount_involved
        description: EUR amount linked to the circular sequence.
        tests:
          - not_null
      - name: description
        description: Human-readable explanation of why the alert fired.
        tests:
          - not_null

  - name: alert_layering
    description: Detects sequential transfer chains with step-down amounts consistent with layering commissions.
    columns:
      - name: alert_id
        description: Deterministic alert identifier.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account that triggered the layering pattern.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_type
        description: AML typology code for the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LAYERING']
      - name: severity
        description: Risk-based severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
        description: Timestamp when the layering chain reached its final hop.
        tests:
          - not_null
      - name: amount_involved
        description: EUR amount linked to the layering chain.
        tests:
          - not_null
      - name: description
        description: Human-readable explanation of why the alert fired.
        tests:
          - not_null

  - name: alert_high_risk_geography
    description: Detects accounts transacting with a hardcoded high-risk jurisdiction watchlist used for
```

```
enhanced due diligence.
    columns:
      - name: alert_id
        description: Deterministic alert identifier.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account exposed to a high-risk geography.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_type
        description: AML typology code for the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['HIGH_RISK_GEOGRAPHY']
      - name: severity
        description: Risk-based severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
        description: Timestamp when the high-risk geography exposure was last observed.
        tests:
          - not_null
      - name: amount_involved
        description: EUR amount linked to the geography exposure.
        tests:
          - not_null
      - name: description
        description: Human-readable explanation of why the alert fired.
        tests:
          - not_null

  - name: fct_alerts
    description: Unioned account-level alert fact table across all AML detection rules.
    columns:
      - name: alert_id
        description: Deterministic alert identifier across all rule models.
        tests:
          - not_null
          - unique
      - name: account_id
        description: Account associated with the alert.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_type
        description: AML typology code associated with the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['SMURFING', 'FAN_OUT', 'FAN_IN', 'CIRCULAR', 'LAYERING', 'HIGH_RISK_GEOGRAPHY']
      - name: severity
        description: Final severity assigned to the alert.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: detected_at
        description: Pattern detection timestamp.
        tests:
          - not_null
      - name: amount_involved
        description: Estimated EUR amount associated with the detection event.
        tests:
          - not_null
      - name: description
        description: Human-readable rationale for the alert.
        tests:
          - not_null

  - name: fct_sar_candidates
    description: Accounts recommended for SAR review because they breached multi-alert or CRITICAL severity
thresholds.
    columns:
      - name: account_id
        description: Account recommended for escalation.
        tests:
          - not_null
          - unique
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: alert_count
        description: Number of alerts linked to the candidate account.
        tests:
          - not_null
```

```
      - name: critical_alert_count
        description: Number of CRITICAL alerts linked to the candidate account.
        tests:
          - not_null
      - name: max_severity
        description: Highest alert severity observed for the account.
        tests:
          - not_null
          - accepted_values:
              values: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL']
      - name: alert_types
        description: Comma-delimited list of alert typologies associated with the account.
        tests:
          - not_null
      - name: first_alert_at
        description: Timestamp of the first observed alert for the account.
        tests:
          - not_null
      - name: latest_alert_at
        description: Timestamp of the most recent alert for the account.
        tests:
          - not_null
      - name: total_amount_involved
        description: Sum of EUR amounts across linked alerts.
        tests:
          - not_null
      - name: sar_rationale
        description: Human-readable explanation for SAR escalation.
        tests:
          - not_null
```

`experimental/orchestration/dbt/models/marts/features/fct_account_features.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
{{
  config(
    post_hook=[
      "CREATE INDEX IF NOT EXISTS idx_fct_account_features_account_id ON {{ this }} (account_id)",
      "ANALYZE {{ this }}"
    ]
  )
}}
with account_transactions as (
    select
        sender_account_id as account_id,
        receiver_account_id as counterpart_account_id,
        amount_eur,
        transaction_timestamp_utc,
        is_laundering,
        'SENT' as transaction_direction
    from {{ ref('stg_transactions') }}

    union all

    select
        receiver_account_id as account_id,
        sender_account_id as counterpart_account_id,
        amount_eur,
        transaction_timestamp_utc,
        is_laundering,
        'RECEIVED' as transaction_direction
    from {{ ref('stg_transactions') }}
),
-- dataset_max_timestamp: scalar subquery used as a cross join to compute
-- velocity_7d relative to the dataset horizon rather than wall-clock time.
-- This avoids incorrect zeroing of velocity for historical datasets.
dataset_max_timestamp as (
    select max(transaction_timestamp_utc) as max_transaction_timestamp
    from {{ ref('stg_transactions') }}
),
aggregated as (
    select
        t.account_id,
        round(sum(case when t.transaction_direction = 'SENT' then t.amount_eur else 0 end), 2) as total_sent,
        round(sum(case when t.transaction_direction = 'RECEIVED' then t.amount_eur else 0 end), 2) as
total_received,
        count(*) as n_transactions,
        round(avg(case when t.transaction_direction = 'SENT' then t.amount_eur end), 2) as avg_amount_sent,
        round(coalesce(stddev_samp(case when t.transaction_direction = 'SENT' then t.amount_eur end), 0), 2) as
stddev_amount_sent,
        count(distinct t.counterpart_account_id) as n_unique_counterparts,
        count(distinct counterpart.country) as n_countries_transacted,
        round(
            count(*) filter (
                where extract(hour from t.transaction_timestamp_utc) between 0 and 5
            )::numeric / nullif(count(*), 0),
            4
        ) as ratio_night_transactions,
        round(max(t.amount_eur), 2) as max_single_transaction,
        round(
            sum(
                case
                    when t.transaction_direction = 'SENT'
```

```
                    and t.transaction_timestamp_utc >= d.max_transaction_timestamp - interval '7 days'
                   then t.amount_eur
                   else 0
               end
           ),
           2
       ) as velocity_7d,
       max(t.is_laundering) as is_laundering_any
   from account_transactions t
   left join {{ ref('stg_accounts') }} counterpart
       on t.counterpart_account_id = counterpart.account_id
   cross join dataset_max_timestamp d
   group by t.account_id
)

select
   a.account_id,
   coalesce(f.total_sent, 0) as total_sent,
   coalesce(f.total_received, 0) as total_received,
   coalesce(f.n_transactions, 0) as n_transactions,
   coalesce(f.avg_amount_sent, 0) as avg_amount_sent,
   coalesce(f.stddev_amount_sent, 0) as stddev_amount_sent,
   coalesce(f.n_unique_counterparts, 0) as n_unique_counterparts,
   coalesce(f.n_countries_transacted, 0) as n_countries_transacted,
   coalesce(f.ratio_night_transactions, 0) as ratio_night_transactions,
   coalesce(f.max_single_transaction, 0) as max_single_transaction,
   coalesce(f.velocity_7d, 0) as velocity_7d,
   coalesce(f.is_laundering_any, 0) as is_laundering_any
from {{ ref('stg_accounts') }} a
left join aggregated f
   on a.account_id = f.account_id
```

`experimental/orchestration/dbt/models/marts/features/schema.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
version: 2

models:
  - name: fct_account_features
    description: Account-level AML feature mart combining transactional volume, behavioral patterns, exposure
breadth, and validation labels.
    columns:
      - name: account_id
        description: Unique account identifier from the account master table.
        tests:
          - not_null
          - unique
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: total_sent
        description: Total EUR amount sent by the account across the full dataset.
        tests:
          - not_null
      - name: total_received
        description: Total EUR amount received by the account across the full dataset.
        tests:
          - not_null
      - name: n_transactions
        description: Count of sent and received transactions involving the account.
        tests:
          - not_null
      - name: avg_amount_sent
        description: Average EUR amount across sent transactions only.
        tests:
          - not_null
      - name: stddev_amount_sent
        description: Standard deviation of sent EUR amounts.
        tests:
          - not_null
      - name: n_unique_counterparts
        description: Count of unique counterpart accounts interacting with the account.
        tests:
          - not_null
      - name: n_countries_transacted
        description: Count of distinct counterpart countries linked to the account.
        tests:
          - not_null
      - name: ratio_night_transactions
        description: Share of transactions occurring between 00:00 and 06:00 UTC.
        tests:
          - not_null
      - name: max_single_transaction
        description: Largest single transaction amount involving the account.
        tests:
          - not_null
      - name: velocity_7d
        description: Total EUR amount sent by the account over the latest seven-day observation window.
        tests:
          - not_null
      - name: is_laundering_any
        description: Validation-only target indicating whether the account appears in any positive-labelled
transaction.
```

```
        tests:
          - not_null
          - accepted_values:
              values: [0, 1]
```

`experimental/orchestration/dbt/models/staging/schema.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
version: 2

models:
  - name: stg_transactions
    description: Cleaned and deduplicated transaction feed in UTC with EUR-normalized amounts.
    columns:
      - name: transaction_id
        description: Unique transaction identifier from the source file.
        tests:
          - not_null
          - unique
      - name: transaction_timestamp_utc
        description: Transaction event timestamp normalized to UTC.
        tests:
          - not_null
      - name: sender_account_id
        description: Sending account identifier.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: receiver_account_id
        description: Receiving account identifier.
        tests:
          - not_null
          - relationships:
              to: ref('stg_accounts')
              field: account_id
      - name: amount_original
        description: Original amount in the source transaction currency.
        tests:
          - not_null
      - name: amount_eur
        description: Transaction amount normalized to EUR using the ingestion exchange-rate map.
        tests:
          - not_null
      - name: currency
        description: ISO currency code provided in the source.
        tests:
          - not_null
      - name: transaction_type
        description: Normalized transaction type label.
        tests:
          - not_null
      - name: is_laundering
        description: Synthetic laundering label retained for validation only.
        tests:
          - not_null
          - accepted_values:
              values: [0, 1]
      - name: ingestion_timestamp
        description: UTC timestamp when the row was ingested into PostgreSQL.
        tests:
          - not_null

  - name: stg_accounts
    description: Cleaned and deduplicated account master table.
    columns:
      - name: account_id
        description: Unique account identifier.
        tests:
          - not_null
          - unique
      - name: account_name
        description: Account or customer display name.
        tests:
          - not_null
      - name: country
        description: ISO-style country label for the account domicile.
        tests:
          - not_null
      - name: account_type
        description: Normalized account classification.
        tests:
          - not_null
      - name: ingestion_timestamp
        description: UTC timestamp when the account row was ingested.
        tests:
          - not_null
```

`experimental/orchestration/dbt/models/staging/src_raw.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
version: 2

sources:
  - name: raw
    description: Raw landing tables populated by the AMLGuardian ingestion pipeline.
    database: "{{ env_var('POSTGRES_DB', 'amlguardian') }}"
    schema: raw
    tables:
      - name: transactions
        description: Raw synthetic banking transactions as ingested from the IBM AML dataset.
      - name: accounts
        description: Raw account master data as ingested from the IBM AML dataset.
```

`experimental/orchestration/dbt/models/staging/stg_accounts.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with deduplicated as (
    select distinct on (account_id)
        account_id,
        trim(account_name) as account_name,
        upper(trim(country)) as country,
        upper(trim(account_type)) as account_type,
        ingestion_timestamp
    from {{ source('raw', 'accounts') }}
    where account_id is not null
    order by account_id, ingestion_timestamp desc
)

select
    account_id,
    account_name,
    country,
    account_type,
    ingestion_timestamp
from deduplicated
```

`experimental/orchestration/dbt/models/staging/stg_transactions.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
with deduplicated as (
    select distinct on (transaction_id)
        transaction_id,
        transaction_timestamp_utc,
        sender_account_id,
        receiver_account_id,
        amount_original,
        amount_eur,
        upper(currency) as currency,
        upper(transaction_type) as transaction_type,
        is_laundering::integer as is_laundering,
        ingestion_timestamp
    from {{ source('raw', 'transactions') }}
    where transaction_id is not null
      and sender_account_id is not null
      and receiver_account_id is not null
      and transaction_timestamp_utc is not null
      and amount_eur is not null
    order by transaction_id, ingestion_timestamp desc
)

select
    transaction_id,
    transaction_timestamp_utc,
    sender_account_id,
    receiver_account_id,
    amount_original,
    amount_eur,
    currency,
    transaction_type,
    is_laundering,
    ingestion_timestamp
from deduplicated
```

`experimental/orchestration/dbt/profiles.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
amlguardian:
  target: dev
  outputs:
    dev:
      type: postgres
      host: "{{ env_var('POSTGRES_HOST') }}"
      port: "{{ env_var('POSTGRES_PORT') | as_number }}"
      user: "{{ env_var('POSTGRES_USER') }}"
      pass: "{{ env_var('POSTGRES_PASSWORD') }}"
      dbname: "{{ env_var('POSTGRES_DB') }}"
      schema: analytics
```

```
      threads: 4
```

`experimental/orchestration/dbt/tests/alerts_positive_amounts.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select *
from {{ ref('fct_alerts') }}
where amount_involved <= 0
```

`experimental/orchestration/dbt/tests/features_counterparts_not_exceed_transactions.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select *
from {{ ref('fct_account_features') }}
where n_unique_counterparts > n_transactions
```

`experimental/orchestration/dbt/tests/features_ratio_night_between_zero_and_one.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select *
from {{ ref('fct_account_features') }}
where ratio_night_transactions < 0
   or ratio_night_transactions > 1
```

`experimental/orchestration/dbt/tests/sar_candidates_meet_threshold.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select *
from {{ ref('fct_sar_candidates') }}
where alert_count < 3
  and critical_alert_count = 0
```

`experimental/orchestration/dbt/tests/transactions_amounts_non_negative.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select *
from {{ ref('stg_transactions') }}
where amount_original < 0
   or amount_eur < 0
```

`experimental/orchestration/dbt/tests/transactions_ingestion_not_before_event.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
select *
from {{ ref('stg_transactions') }}
where ingestion_timestamp < transaction_timestamp_utc
```

`experimental/orchestration/docker-compose.yml`

Optional/non-core file retained for archived advanced work or future improvements.

```
services:
  postgres:
    image: postgres:15.6-alpine
    container_name: amlguardian-postgres
    restart: unless-stopped
    env_file:
      - .env
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "${POSTGRES_PORT}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./docker/postgres/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 10

  pgadmin:
    image: dpage/pgadmin4:8.6
    container_name: amlguardian-pgadmin
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
    env_file:
      - .env
```

```
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
      PGADMIN_CONFIG_SERVER_MODE: "False"
      PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED: "False"
      PGADMIN_SERVER_JSON_FILE: /tmp/servers.json
    entrypoint: ["/bin/sh", "/pgadmin4/custom-entrypoint.sh"]
    ports:
      - "${PGADMIN_PORT}:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin
      - ./docker/pgadmin/custom-entrypoint.sh:/pgadmin4/custom-entrypoint.sh:ro

  neo4j:
    image: neo4j:5.20.0-community
    container_name: amlguardian-neo4j
    restart: unless-stopped
    environment:
      NEO4J_AUTH: ${NEO4J_USERNAME}/${NEO4J_PASSWORD}
      NEO4J_PLUGINS: '["graph-data-science"]'
      NEO4J_dbms_security_procedures_unrestricted: gds.*
      NEO4J_server_memory_heap_initial__size: 512M
      NEO4J_server_memory_heap_max__size: 2G
      NEO4J_server_memory_pagecache_size: 1G
    ports:
      - "${NEO4J_HTTP_PORT}:7474"
      - "${NEO4J_BOLT_PORT}:7687"
    volumes:
      - neo4j_data:/data
      - neo4j_logs:/logs
      - neo4j_plugins:/plugins
    healthcheck:
      test: ["CMD-SHELL", "cypher-shell -u ${NEO4J_USERNAME} -p ${NEO4J_PASSWORD} 'RETURN 1;' || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 20


  airflow-init:
    build:
      context: .
      dockerfile: docker/airflow/Dockerfile
    container_name: amlguardian-airflow-init
    restart: "no"
    depends_on:
      postgres:
        condition: service_healthy
    env_file:
      - .env
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__CORE__LOAD_EXAMPLES: "False"
      AIRFLOW__CORE__FERNET_KEY: ${AIRFLOW_FERNET_KEY}
      AIRFLOW__WEBSERVER__SECRET_KEY: ${AIRFLOW_WEBSERVER_SECRET_KEY}
      AIRFLOW__API__AUTH_BACKENDS: airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: "True"
      PYTHONPATH: /opt/airflow:/opt/amlguardian
    user: "${AIRFLOW_UID}:0"
    volumes:
      - airflow_logs:/opt/airflow/logs
      - ./dags:/opt/airflow/dags
      - ./scripts:/opt/amlguardian/scripts
      - ./dbt:/opt/amlguardian/dbt
      - ./sql:/opt/amlguardian/sql
      - ./data:/opt/amlguardian/data
      - ./outputs:/opt/amlguardian/outputs
      - ./.env:/opt/amlguardian/.env:ro
    command: >
      bash -c "mkdir -p /opt/airflow/logs /opt/airflow/dags /opt/amlguardian/outputs
      && airflow db migrate
      && (airflow users delete --username ${AIRFLOW_ADMIN_USERNAME} || true)
      && airflow users create --username ${AIRFLOW_ADMIN_USERNAME} --password ${AIRFLOW_ADMIN_PASSWORD}
--firstname AML --lastname Guardian --role Admin --email ${AIRFLOW_ADMIN_EMAIL}"

  airflow-webserver:
    build:
      context: .
      dockerfile: docker/airflow/Dockerfile
    container_name: amlguardian-airflow-webserver
    restart: unless-stopped
    depends_on:
      airflow-init:
        condition: service_completed_successfully
      postgres:
        condition: service_healthy
    env_file:
      - .env
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__CORE__LOAD_EXAMPLES: "False"
      AIRFLOW__CORE__FERNET_KEY: ${AIRFLOW_FERNET_KEY}
      AIRFLOW__WEBSERVER__SECRET_KEY: ${AIRFLOW_WEBSERVER_SECRET_KEY}
      AIRFLOW__API__AUTH_BACKENDS: airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session
```

```
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: "True"
      PYTHONPATH: /opt/airflow:/opt/amlguardian
    user: "${AIRFLOW_UID}:0"
    ports:
      - "${AIRFLOW_PORT}:8080"
    volumes:
      - airflow_logs:/opt/airflow/logs
      - ./dags:/opt/airflow/dags
      - ./scripts:/opt/amlguardian/scripts
      - ./dbt:/opt/amlguardian/dbt
      - ./sql:/opt/amlguardian/sql
      - ./data:/opt/amlguardian/data
      - ./outputs:/opt/amlguardian/outputs
      - ./.env:/opt/amlguardian/.env:ro
    command: webserver

  airflow-scheduler:
    build:
      context: .
      dockerfile: docker/airflow/Dockerfile
    container_name: amlguardian-airflow-scheduler
    restart: unless-stopped
    depends_on:
      airflow-init:
        condition: service_completed_successfully
      postgres:
        condition: service_healthy
    env_file:
      - .env
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__CORE__LOAD_EXAMPLES: "False"
      AIRFLOW__CORE__FERNET_KEY: ${AIRFLOW_FERNET_KEY}
      AIRFLOW__WEBSERVER__SECRET_KEY: ${AIRFLOW_WEBSERVER_SECRET_KEY}
      AIRFLOW__API__AUTH_BACKENDS: airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
postgresql+psycopg2://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: "True"
      PYTHONPATH: /opt/airflow:/opt/amlguardian
    user: "${AIRFLOW_UID}:0"
    volumes:
      - airflow_logs:/opt/airflow/logs
      - ./dags:/opt/airflow/dags
      - ./scripts:/opt/amlguardian/scripts
      - ./dbt:/opt/amlguardian/dbt
      - ./sql:/opt/amlguardian/sql
      - ./data:/opt/amlguardian/data
      - ./outputs:/opt/amlguardian/outputs
      - ./.env:/opt/amlguardian/.env:ro
    command: scheduler

volumes:
  postgres_data:
  pgadmin_data:
  airflow_logs:
  neo4j_data:
  neo4j_logs:
  neo4j_plugins:
```

`experimental/orchestration/docker/postgres/init/01_init.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS features;
CREATE SCHEMA IF NOT EXISTS alerts;
CREATE SCHEMA IF NOT EXISTS ml;

CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

`experimental/orchestration/docker/postgres/init/02_post_staging_indexes.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
-- Post-staging indexes for AMLGuardian
--
-- These indexes target tables created by dbt staging models (staging.stg_transactions,
-- staging.stg_accounts). They do NOT exist at container init time and CANNOT be
-- applied by 01_init.sql. Run this file manually after the first dbt staging run:
--
--   docker exec -i amlguardian-postgres psql -U $POSTGRES_USER -d $POSTGRES_DB \
--     < docker/postgres/init/02_post_staging_indexes.sql
--
-- Alternatively, attach these as dbt post-hooks on the staging models if the
-- project grows to warrant it. All statements are idempotent (IF NOT EXISTS).

-- Support fct_account_features GROUP BY account_id and velocity_7d window filter
CREATE INDEX IF NOT EXISTS idx_stg_transactions_sender_ts
    ON staging.stg_transactions (sender_account_id, transaction_timestamp_utc);
CREATE INDEX IF NOT EXISTS idx_stg_transactions_receiver_ts
```

```
    ON staging.stg_transactions (receiver_account_id, transaction_timestamp_utc);

-- Support the alert_lookback_days base-case filter on both recursive CTE models
CREATE INDEX IF NOT EXISTS idx_stg_transactions_ts
    ON staging.stg_transactions (transaction_timestamp_utc);

-- Support left joins from fct_account_features and alert models into stg_accounts
CREATE INDEX IF NOT EXISTS idx_stg_accounts_id
    ON staging.stg_accounts (account_id);
```

`experimental/orchestration/legacy_postgres_ingestion/common.py`

Optional/non-core file retained for archived advanced work or future improvements.

```
from __future__ import annotations

import logging
import os
from pathlib import Path

from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.engine import Engine


PROJECT_ROOT = Path(__file__).resolve().parents[1]
ENV_PATH = PROJECT_ROOT / ".env"

if ENV_PATH.exists():
    load_dotenv(ENV_PATH)
else:
    load_dotenv()


def configure_logging(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        handler = logging.StreamHandler()
        handler.setFormatter(
            logging.Formatter(
                fmt="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
                datefmt="%Y-%m-%d %H:%M:%S",
            )
        )
        logger.addHandler(handler)
    logger.setLevel(os.getenv("LOG_LEVEL", "INFO").upper())
    logger.propagate = False
    return logger


def env(name: str, default: str | None = None, required: bool = False) -> str:
    value = os.getenv(name, default)
    if required and not value:
        raise RuntimeError(f"Missing required environment variable: {name}")
    return value or ""


def get_postgres_url() -> str:
    user = env("POSTGRES_USER", required=True)
    password = env("POSTGRES_PASSWORD", required=True)
    host = env("POSTGRES_HOST", "localhost")
    port = env("POSTGRES_PORT", "5432")
    database = env("POSTGRES_DB", required=True)
    return f"postgresql+psycopg2://{user}:{password}@{host}:{port}/{database}"


def get_engine() -> Engine:
    return create_engine(get_postgres_url(), future=True)


def ensure_directory(path: str | Path) -> Path:
    directory = Path(path)
    directory.mkdir(parents=True, exist_ok=True)
    return directory
```

`experimental/orchestration/legacy_postgres_ingestion/ingest.py`

Optional/non-core file retained for archived advanced work or future improvements.

```
from __future__ import annotations

import re
from datetime import UTC, datetime
from io import StringIO
from pathlib import Path

import pandas as pd
import psycopg2

from common import configure_logging, env


LOGGER = configure_logging("amlguardian.ingest")
```

```
CHUNK_SIZE = 100_000
FX_RATES = {
    "EUR": 1.00,
    "USD": 0.92,
    "GBP": 1.17,
    "CHF": 1.04,
}


def postgres_connection():
    return psycopg2.connect(
        host=env("POSTGRES_HOST", "localhost"),
        port=env("POSTGRES_PORT", "5432"),
        dbname=env("POSTGRES_DB", required=True),
        user=env("POSTGRES_USER", required=True),
        password=env("POSTGRES_PASSWORD", required=True),
    )


def normalize_headers(columns: list[str]) -> list[str]:
    normalized = []
    for column in columns:
        column = re.sub(r"([a-z0-9])([A-Z])", r"\1_\2", column)
        column = re.sub(r"[^a-zA-Z0-9]+", "_", column).strip("_")
        normalized.append(column.lower())
    return normalized


def normalize_currency(value: object) -> str:
    if value is None or (isinstance(value, float) and pd.isna(value)):
        return "EUR"
    cleaned = str(value).strip().upper()
    return cleaned or "EUR"


def clean_transactions(chunk: pd.DataFrame, ingested_at: datetime) -> pd.DataFrame:
    chunk = chunk.copy()
    chunk.columns = normalize_headers(chunk.columns.tolist())

    rename_map = {
        "transaction_id": "transaction_id",
        "timestamp": "transaction_timestamp_utc",
        "sender_account_id": "sender_account_id",
        "receiver_account_id": "receiver_account_id",
        "amount": "amount_original",
        "currency": "currency",
        "transaction_type": "transaction_type",
        "is_laundering": "is_laundering",
    }
    chunk = chunk.rename(columns=rename_map)

    required_columns = [
        "transaction_id",
        "transaction_timestamp_utc",
        "sender_account_id",
        "receiver_account_id",
        "amount_original",
        "currency",
        "transaction_type",
        "is_laundering",
    ]
    for column in required_columns:
        if column not in chunk.columns:
            raise ValueError(f"Missing expected transaction column: {column}")

    chunk["transaction_id"] = chunk["transaction_id"].astype(str).str.strip()
    chunk["sender_account_id"] = chunk["sender_account_id"].astype(str).str.strip()
    chunk["receiver_account_id"] = chunk["receiver_account_id"].astype(str).str.strip()
    chunk["currency"] = chunk["currency"].apply(normalize_currency)
    chunk["transaction_type"] = (
        chunk["transaction_type"].fillna("UNKNOWN").astype(str).str.strip().str.upper()
    )

    chunk["amount_original"] = pd.to_numeric(chunk["amount_original"], errors="coerce")
    chunk["amount_eur"] = (
        chunk["amount_original"] * chunk["currency"].map(FX_RATES).fillna(1.0)
    ).round(2)
    chunk["transaction_timestamp_utc"] = pd.to_datetime(
        chunk["transaction_timestamp_utc"], errors="coerce", utc=True
    )
    chunk["is_laundering"] = (
        pd.to_numeric(chunk["is_laundering"], errors="coerce")
        .fillna(0)
        .clip(lower=0, upper=1)
        .astype(int)
    )
    chunk["ingestion_timestamp"] = ingested_at

    before = len(chunk)
    chunk = chunk.dropna(
        subset=[
            "transaction_timestamp_utc",
            "amount_original",
            "amount_eur",
        ]
```

```
    )
    chunk = chunk[
        (chunk["transaction_id"] != "")
        & (chunk["sender_account_id"] != "")
        & (chunk["receiver_account_id"] != "")
    ]
    chunk = chunk.drop_duplicates(subset=["transaction_id"])
    dropped = before - len(chunk)
    if dropped:
        LOGGER.info("Dropped %s invalid or duplicate transaction rows in current chunk", dropped)

    return chunk[
        [
            "transaction_id",
            "transaction_timestamp_utc",
            "sender_account_id",
            "receiver_account_id",
            "amount_original",
            "amount_eur",
            "currency",
            "transaction_type",
            "is_laundering",
            "ingestion_timestamp",
        ]
    ]


def clean_accounts(accounts: pd.DataFrame, ingested_at: datetime) -> pd.DataFrame:
    accounts = accounts.copy()
    accounts.columns = normalize_headers(accounts.columns.tolist())
    rename_map = {
        "account_id": "account_id",
        "name": "account_name",
        "country": "country",
        "account_type": "account_type",
    }
    accounts = accounts.rename(columns=rename_map)

    required_columns = ["account_id", "account_name", "country", "account_type"]
    for column in required_columns:
        if column not in accounts.columns:
            raise ValueError(f"Missing expected account column: {column}")

    accounts["account_id"] = accounts["account_id"].astype(str).str.strip()
    accounts["account_name"] = accounts["account_name"].fillna("UNKNOWN").astype(str).str.strip()
    accounts["country"] = accounts["country"].fillna("UNKNOWN").astype(str).str.strip().str.upper()
    accounts["account_type"] = (
        accounts["account_type"].fillna("UNKNOWN").astype(str).str.strip().str.upper()
    )
    accounts["ingestion_timestamp"] = ingested_at

    before = len(accounts)
    accounts = accounts[accounts["account_id"] != ""].drop_duplicates(subset=["account_id"])
    dropped = before - len(accounts)
    if dropped:
        LOGGER.info("Dropped %s invalid or duplicate account rows", dropped)

    return accounts[required_columns + ["ingestion_timestamp"]]


def copy_dataframe(cursor, dataframe: pd.DataFrame, table_name: str, columns: list[str]) -> None:
    buffer = StringIO()
    dataframe.to_csv(buffer, index=False, header=False, na_rep="\\N")
    buffer.seek(0)
    quoted_columns = ", ".join(columns)
    cursor.copy_expert(
        f"COPY {table_name} ({quoted_columns}) FROM STDIN WITH (FORMAT CSV, NULL '\\N')",
        buffer,
    )


def prepare_raw_tables(connection) -> None:
    ddl = """
    CREATE SCHEMA IF NOT EXISTS raw;

    DROP TABLE IF EXISTS raw.transactions;
    CREATE TABLE raw.transactions (
        transaction_id TEXT,
        transaction_timestamp_utc TIMESTAMPTZ,
        sender_account_id TEXT,
        receiver_account_id TEXT,
        amount_original NUMERIC(18, 2),
        amount_eur NUMERIC(18, 2),
        currency TEXT,
        transaction_type TEXT,
        is_laundering INTEGER,
        ingestion_timestamp TIMESTAMPTZ
    );

    DROP TABLE IF EXISTS raw.accounts;
    CREATE TABLE raw.accounts (
        account_id TEXT,
        account_name TEXT,
        country TEXT,
        account_type TEXT,
```

```
        ingestion_timestamp TIMESTAMPTZ
    );
    """
    with connection.cursor() as cursor:
        cursor.execute(ddl)
    connection.commit()


def post_load_housekeeping(connection) -> None:
    statements = [
        """
        DELETE FROM raw.transactions a
        USING raw.transactions b
        WHERE a.ctid < b.ctid
          AND a.transaction_id = b.transaction_id;
        """,
        """
        DELETE FROM raw.accounts a
        USING raw.accounts b
        WHERE a.ctid < b.ctid
          AND a.account_id = b.account_id;
        """,
        """
        CREATE INDEX IF NOT EXISTS idx_raw_transactions_id ON raw.transactions (transaction_id);
        CREATE INDEX IF NOT EXISTS idx_raw_transactions_sender_ts
            ON raw.transactions (sender_account_id, transaction_timestamp_utc);
        CREATE INDEX IF NOT EXISTS idx_raw_transactions_receiver_ts
            ON raw.transactions (receiver_account_id, transaction_timestamp_utc);
        CREATE INDEX IF NOT EXISTS idx_raw_accounts_id ON raw.accounts (account_id);
        """,
    ]
    with connection.cursor() as cursor:
        for statement in statements:
            cursor.execute(statement)
    connection.commit()


def ingest_accounts(connection, accounts_path: Path, ingested_at: datetime) -> int:
    LOGGER.info("Reading accounts from %s", accounts_path)
    accounts = pd.read_csv(accounts_path)
    cleaned = clean_accounts(accounts, ingested_at)
    with connection.cursor() as cursor:
        copy_dataframe(
            cursor,
            cleaned,
            "raw.accounts",
            ["account_id", "account_name", "country", "account_type", "ingestion_timestamp"],
        )
    connection.commit()
    LOGGER.info("Loaded %s account rows into raw.accounts", len(cleaned))
    return len(cleaned)


def ingest_transactions(connection, transactions_path: Path, ingested_at: datetime) -> int:
    LOGGER.info("Reading transactions from %s in chunks of %s", transactions_path, CHUNK_SIZE)
    total_rows = 0
    for chunk_number, chunk in enumerate(pd.read_csv(transactions_path, chunksize=CHUNK_SIZE), start=1):
        cleaned = clean_transactions(chunk, ingested_at)
        with connection.cursor() as cursor:
            copy_dataframe(
                cursor,
                cleaned,
                "raw.transactions",
                [
                    "transaction_id",
                    "transaction_timestamp_utc",
                    "sender_account_id",
                    "receiver_account_id",
                    "amount_original",
                    "amount_eur",
                    "currency",
                    "transaction_type",
                    "is_laundering",
                    "ingestion_timestamp",
                ],
            )
        connection.commit()
        total_rows += len(cleaned)
        LOGGER.info(
            "Processed chunk %s | cumulative transactions loaded: %s",
            chunk_number,
            total_rows,
        )
    return total_rows


def main() -> None:
    transactions_path = Path(env("TRANSACTIONS_CSV", required=True))
    accounts_path = Path(env("ACCOUNTS_CSV", required=True))
    ingested_at = datetime.now(tz=UTC)

    if not transactions_path.exists():
        raise FileNotFoundError(f"Transactions CSV not found: {transactions_path}")
    if not accounts_path.exists():
        raise FileNotFoundError(f"Accounts CSV not found: {accounts_path}")
```

```
    LOGGER.info("Starting raw ingestion for AMLGuardian")
    connection = postgres_connection()
    try:
        prepare_raw_tables(connection)
        account_rows = ingest_accounts(connection, accounts_path, ingested_at)
        transaction_rows = ingest_transactions(connection, transactions_path, ingested_at)
        post_load_housekeeping(connection)
        LOGGER.info(
            "Ingestion finished successfully | accounts=%s | transactions=%s",
            account_rows,
            transaction_rows,
        )
    finally:
        connection.close()


if __name__ == "__main__":
    main()
```

`experimental/sql_investigations/investigations/circular_flows.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
/*
Purpose:
Reconstruct complete circular transaction chains of the form A -> B -> C -> A within seven days.

Why it matters:
Circular movement is a strong layering indicator because funds are routed through multiple parties and returned
to the origin to obscure provenance and beneficial ownership.
*/

with recursive paths as (
    select
        t.sender_account_id as root_account_id,
        t.receiver_account_id as current_account_id,
        array[t.sender_account_id, t.receiver_account_id]::text[] as account_path,
        array[t.transaction_id]::text[] as transaction_path,
        t.transaction_timestamp_utc as start_ts,
        t.transaction_timestamp_utc as last_ts,
        t.amount_eur as total_amount,
        1 as depth
    from staging.stg_transactions t
    where t.sender_account_id <> t.receiver_account_id

    union all

    select
        p.root_account_id,
        t.receiver_account_id,
        p.account_path || t.receiver_account_id,
        p.transaction_path || t.transaction_id,
        p.start_ts,
        t.transaction_timestamp_utc,
        p.total_amount + t.amount_eur,
        p.depth + 1
    from paths p
    join staging.stg_transactions t
        on t.sender_account_id = p.current_account_id
       and t.transaction_timestamp_utc >= p.last_ts
       and t.transaction_timestamp_utc <= p.start_ts + interval '7 days'
    where p.depth < 3
      and (
            (p.depth = 2 and t.receiver_account_id = p.root_account_id)
         or (p.depth < 2 and not t.receiver_account_id = any(p.account_path))
      )
)

select
    root_account_id,
    account_path[1] as account_a,
    account_path[2] as account_b,
    account_path[3] as account_c,
    transaction_path[1] as tx_ab,
    transaction_path[2] as tx_bc,
    transaction_path[3] as tx_ca,
    start_ts as first_leg_timestamp,
    last_ts as final_leg_timestamp,
    round(total_amount, 2) as circular_amount
from paths
where depth = 3
  and current_account_id = root_account_id
  and cardinality(account_path) = 4
order by circular_amount desc, final_leg_timestamp desc
```

`experimental/sql_investigations/investigations/fan_out_investigation.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
/*
Purpose:
Expose accounts that disperse funds across many beneficiaries and show where the money was distributed within
the alert window.
```

```
Why it matters:
Fan-out behavior is common in mule-account networks, payout hubs, and layering stages where illicit proceeds are
 quickly fragmented after receipt.
*/

with fan_out_windows as (
    select
        account_id,
        detected_at,
        amount_involved,
        detected_at - interval '24 hours' as window_start
    from alerts.alert_fan_out
),
distribution as (
    select
        f.account_id,
        f.window_start,
        f.detected_at,
        t.receiver_account_id as beneficiary_account_id,
        acc.country as beneficiary_country,
        count(*) as txn_count,
        round(sum(t.amount_eur), 2) as total_amount,
        round(avg(t.amount_eur), 2) as avg_amount
    from fan_out_windows f
    join staging.stg_transactions t
        on t.sender_account_id = f.account_id
       and t.transaction_timestamp_utc between f.window_start and f.detected_at
    left join staging.stg_accounts acc
        on t.receiver_account_id = acc.account_id
    group by 1, 2, 3, 4, 5
)

select
    account_id,
    window_start,
    detected_at,
    beneficiary_account_id,
    beneficiary_country,
    txn_count,
    total_amount,
    avg_amount,
    dense_rank() over (
        partition by account_id, detected_at
        order by total_amount desc
    ) as beneficiary_rank_by_amount
from distribution
order by account_id, detected_at desc, total_amount desc
```

`*experimental/sql_investigations/investigations/geographic_analysis.sql*`

Optional/non-core file retained for archived advanced work or future improvements.

```
/*
Purpose:
Measure transaction corridors by country pair and flag routes that touch a hardcoded high-risk jurisdiction
watchlist.

Why it matters:
Cross-border concentration, especially involving monitored or high-risk jurisdictions, is a key AML risk
indicator and helps analysts focus on suspicious geographies and payment corridors.
*/

with high_risk_countries as (
    select country
    from (
        values
            ('IRAN'),
            ('DPRK'),
            ('MYANMAR'),
            ('SYRIA'),
            ('YEMEN'),
            ('HAITI'),
            ('SOUTH SUDAN'),
            ('NIGERIA'),
            ('VENEZUELA'),
            ('CAMEROON'),
            ('CROATIA'),
            ('KENYA'),
            ('NAMIBIA'),
            ('VIETNAM'),
            ('SOUTH AFRICA')
    ) as watchlist(country)
)

select
    sender.country as sender_country,
    receiver.country as receiver_country,
    count(*) as txn_count,
    round(sum(t.amount_eur), 2) as total_amount_eur,
    round(avg(t.amount_eur), 2) as avg_amount_eur,
    count(*) filter (where t.is_laundering = 1) as labeled_laundering_txn_count,
    case
        when sender.country in (select country from high_risk_countries)
          or receiver.country in (select country from high_risk_countries)
```

```
        then 'HIGH_RISK_CORRIDOR'
        else 'STANDARD_CORRIDOR'
    end as corridor_flag
from staging.stg_transactions t
left join staging.stg_accounts sender
    on t.sender_account_id = sender.account_id
left join staging.stg_accounts receiver
    on t.receiver_account_id = receiver.account_id
group by 1, 2, 7
order by corridor_flag desc, total_amount_eur desc, txn_count desc
```

`experimental/sql_investigations/investigations/high_risk_accounts.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
/*
Purpose:
Prioritize accounts that combine multiple alert typologies with a high machine-learning risk score.

Why it matters:
Accounts that trigger diverse rule-based indicators and also score highly in the behavioral model warrant
accelerated review because they exhibit both known typologies and anomalous aggregate behavior.
*/

with alert_summary as (
    select
        account_id,
        count(*) as alert_count,
        count(distinct alert_type) as alert_type_count,
        string_agg(distinct alert_type, ', ' order by alert_type) as alert_types,
        max(case severity
            when 'CRITICAL' then 4
            when 'HIGH' then 3
            when 'MEDIUM' then 2
            else 1
        end) as max_severity_rank,
        round(sum(amount_involved), 2) as total_alert_amount
    from alerts.fct_alerts
    group by account_id
)

select
    a.account_id,
    acc.account_name,
    acc.country,
    acc.account_type,
    a.alert_count,
    a.alert_type_count,
    a.alert_types,
    case a.max_severity_rank
        when 4 then 'CRITICAL'
        when 3 then 'HIGH'
        when 2 then 'MEDIUM'
        else 'LOW'
    end as max_alert_severity,
    a.total_alert_amount,
    r.risk_score,
    r.risk_tier,
    r.top_3_features
from alert_summary a
join ml.risk_scores r
    on a.account_id = r.account_id
left join staging.stg_accounts acc
    on a.account_id = acc.account_id
where a.alert_type_count >= 2
  and r.risk_score >= 70
order by r.risk_score desc, a.alert_count desc, a.total_alert_amount desc
```

`experimental/sql_investigations/investigations/sar_candidates.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
/*
Purpose:
Assemble a full escalation pack for SAR review candidates, combining account master data, alert activity, ML
score, graph metrics, and transaction statistics.

Why it matters:
Analysts need a single, investigation-ready view to document suspicion, assess materiality, and decide whether a
 case should move to formal SAR drafting.
*/

with transaction_stats as (
    select
        account_id,
        count(*) as n_transactions,
        round(sum(amount_eur), 2) as total_transaction_amount,
        round(max(amount_eur), 2) as max_transaction_amount,
        max(transaction_timestamp_utc) as last_transaction_at
    from (
        select sender_account_id as account_id, amount_eur, transaction_timestamp_utc
        from staging.stg_transactions
        union all
```

```
        select receiver_account_id as account_id, amount_eur, transaction_timestamp_utc
        from staging.stg_transactions
    ) t
    group by account_id
),
alert_detail as (
    select
        account_id,
        string_agg(alert_type || ' [' || severity || ']', ', ' order by detected_at desc) as alert_timeline
    from alerts.fct_alerts
    group by account_id
)

select
    s.account_id,
    acc.account_name,
    acc.country,
    acc.account_type,
    s.alert_count,
    s.critical_alert_count,
    s.max_severity,
    s.alert_types,
    s.sar_rationale,
    s.first_alert_at,
    s.latest_alert_at,
    s.total_amount_involved,
    r.risk_score,
    r.risk_tier,
    r.top_3_features,
    g.pagerank_score,
    g.community_id,
    ts.n_transactions,
    ts.total_transaction_amount,
    ts.max_transaction_amount,
    ts.last_transaction_at,
    ad.alert_timeline
from alerts.fct_sar_candidates s
left join staging.stg_accounts acc
    on s.account_id = acc.account_id
left join ml.risk_scores r
    on s.account_id = r.account_id
left join ml.graph_account_metrics g
    on s.account_id = g.account_id
left join transaction_stats ts
    on s.account_id = ts.account_id
left join alert_detail ad
    on s.account_id = ad.account_id
order by s.critical_alert_count desc, r.risk_score desc nulls last, g.pagerank_score desc nulls last
```

`experimental/sql_investigations/investigations/smurfing_deep_dive.sql`

Optional/non-core file retained for archived advanced work or future improvements.

```
/*
Purpose:
Investigate structuring (smurfing) alerts by reconstructing the suspicious timeline around the detected 72-hour
window.

Why it matters:
Repeated payments just below reporting or control thresholds are a classic AML red flag because they indicate
deliberate evasion of monitoring controls.
*/

with alert_windows as (
    select
        account_id,
        detected_at,
        amount_involved,
        description,
        detected_at - interval '72 hours' as window_start
    from alerts.alert_smurfing
),
timeline as (
    select
        a.account_id,
        a.detected_at as alert_detected_at,
        a.window_start,
        t.transaction_id,
        t.transaction_timestamp_utc,
        t.receiver_account_id as beneficiary_account_id,
        t.amount_eur,
        t.currency,
        t.transaction_type,
        t.is_laundering
    from alert_windows a
    join staging.stg_transactions t
        on t.sender_account_id = a.account_id
       and t.transaction_timestamp_utc between a.window_start and a.detected_at
       and t.amount_eur < 9999
)

select
    account_id,
    alert_detected_at,
```

```
    window_start,
    transaction_id,
    transaction_timestamp_utc,
    beneficiary_account_id,
    amount_eur,
    currency,
    transaction_type,
    is_laundering,
    count(*) over (
        partition by account_id, alert_detected_at
        order by transaction_timestamp_utc
        rows between unbounded preceding and current row
    ) as cumulative_sub_threshold_txn_count,
    sum(amount_eur) over (
        partition by account_id, alert_detected_at
        order by transaction_timestamp_utc
        rows between unbounded preceding and current row
    ) as cumulative_sub_threshold_amount
from timeline
order by account_id, alert_detected_at desc, transaction_timestamp_utc
```

10. Interview Defense Notes

60-second explanation

This is a simplified AML analytics portfolio project using synthetic data. It loads account and transaction CSVs, cleans them with Python/Pandas, creates behavioural features, applies a small set of explainable rules, and outputs an alerts table for analyst review or dashboarding. It is not a compliance decision system; it is a practical project to demonstrate data cleaning, feature engineering, SQL thinking and AML monitoring concepts at junior level.

Technical decisions

- Kept the main path CSV/Pandas based so it is easy to run and explain.

- Used explainable rules instead of ML as the core detection approach.

- Generated both account-level and transaction-level features.

- Kept SQL examples for interview discussion and BI/database practice.

- Archived advanced work under experimental/ instead of deleting it.

Trade-offs

- Simplicity over orchestration: easier to understand, but not scheduled.

- Fixed thresholds over adaptive models: explainable, but less flexible.

- Synthetic data over real data: safe for a portfolio, but less realistic.

- Rule-based alerts over final decisions: appropriate for education, but limited for real compliance.

Why the project was simplified

The simplified version is better aligned with junior Data Analyst, BI Analyst and AML Analytics Junior roles. It shows the author understands the data workflow and AML monitoring logic without overstating production readiness.

Limitations

- No real customer due diligence data.

- No analyst feedback loop.

- No live regulatory watchlist refresh.

- No transaction monitoring calibration process.

- No production controls, access management or audit workflow.

Future improvements

- Add a small Streamlit or Power BI dashboard.

- Add configurable rule thresholds.

- Add analyst review status fields.

- Move the SQL examples into SQLite/PostgreSQL exercises.

- Revisit experimental/ for orchestration, graph analytics or ML only after the core story is solid.

Likely interview questions and strong answers

Q: Is this a real AML system? A: No. It is an educational portfolio project using synthetic data. It generates explainable alerts for practice, but it does not make compliance decisions.

Q: Why rule-based detection instead of ML? A: For junior analytics roles, rule-based detection is easier to explain, audit and connect to AML typologies. ML can be future work, but the main learning goal is clean data and transparent alert logic.

Q: What would you improve first? A: I would add configurable thresholds, a small dashboard and an analyst feedback field so reviewed alerts can be tracked.

Q: What does severity mean here? A: It is a simple priority label based on rule metrics, not a final risk rating. It helps order alerts for review.

Q: How would this differ in a real institution? A: A real institution would require governance, threshold calibration, KYC context, alert investigation workflow, audit trails, model/rule validation and regulatory procedures.

11. Glossary

- **AML:** Anti-Money Laundering; controls and analysis used to identify and investigate suspicious financial activity.

- **Transaction monitoring:** Review of transaction activity to detect unusual or potentially suspicious patterns.

- **Alert:** A record generated by a rule or model that requires review.

- **Smurfing:** Splitting funds into multiple smaller transactions, often discussed as structuring.

- **Fan-in:** Many senders transferring into one receiver within a short period.

- **Fan-out:** One sender transferring to many receivers within a short period.

- **High-risk geography:** Activity involving a country or jurisdiction that requires additional context or review.

- **Synthetic data:** Artificial data created for learning or testing, not real customer data.

- **Feature engineering:** Creating analytical variables from raw data, such as counts, totals or flags.

- **Severity:** A simple priority label for an alert in this project.

- **False positive:** An alert that looks unusual by rule logic but is later explained as legitimate.

- **Analyst review:** Human investigation step where context, customer profile and evidence are assessed.

Skipped Files

The following files were skipped to avoid heavy, binary, irrelevant or sensitive content.

- .claude/: excluded directory: .claude

- .env: local environment file excluded to avoid exporting secrets

- .git/: excluded directory: .git

- .tools/: excluded directory: .tools

- aml-interview-demo/: excluded directory: aml-interview-demo

- data/.gitkeep: not an included text type

- data/processed/.gitkeep: not an included text type

- docs/export/AML_project_full_study_pack.md: generated export output

- docs/export/AML_project_full_study_pack.pdf: excluded binary/heavy suffix: .pdf

- experimental/archive_docs/AMLGuardian_Guia_Completa.pdf: excluded binary/heavy suffix: .pdf

- experimental/archive_docs/legacy_project_dossier.html: not an included text type

- experimental/orchestration/.dockerignore: not an included text type
- experimental/orchestration/dags/__pycache__/: excluded directory: __pycache__
- experimental/orchestration/docker/airflow/Dockerfile: not an included text type
- experimental/orchestration/docker/pgadmin/custom-entrypoint.sh: not an included text type
- experimental/orchestration/legacy_postgres_ingestion/__pycache__/: excluded directory: __pycache__
- outputs/.gitkeep: not an included text type
- src/__pycache__/: excluded directory: __pycache__